

OBJECT ORIENTED PROGRAMMING

CDT2001

by
Ms. Titiksha Bhagat
CSE- Data Science, RCOEM
bhagattt@rcoem.edu.in

UNIT 2

Syllabus

Concept of overloading: Constructor Overloading, Function Overloading. Arrays and Array of objects, Wrapper classes (Integer , Double etc.), String Class, creating packages, importing packages. Lambda Expressions Introduction, Block, Passing Lambda expression as Argument

Constructor Overloading

3

- ❑ Multiple Constructors in same class with same name but different arguments
- ❑ Provides alternate ways to create objects of a class
- ❑ JVM differentiates between constructors on the basis of argument list

Overloading Constructor: Example

```
class Box {  
    double width;  
    double height;  
    double depth;  
    // This is the constructor for Box.  
    Box(double w, double h, double d){  
        width = w;  
        height = h;  
        depth = d;  
    }  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
}
```

Overloading Constructor: Example Discussion

- ❧ As you can see, the `Box( )` constructor requires three parameters. This means that all declarations of `Box` objects must pass three arguments to the `Box( )` constructor. For example, the following statement is currently invalid:

```
Box ob = new Box();
```
- ❧ Since `Box( )` requires three arguments, it's an error to call it without them. This raises some important questions. What if you simply wanted a box and did not care (or know) what its initial dimensions were? Or, what if you want to be able to initialize a cube by specifying only one value that would be used for all three dimensions? As the `Box` class is currently written, these other options are not available to you.
- ❧ Fortunately, the solution to these problems is quite easy: simply overload the `Box` constructor so that it handles the situations just described

Overloading Constructor

- ❧ As you can see, the `Box( )` constructor requires three parameters. This means that all declarations of `Box` objects must pass three arguments to the `Box( )` constructor. For example, the following statement is currently invalid:

```
Box ob = new Box();
```
- ❧ Since `Box( )` requires three arguments, it's an error to call it without them. This raises some important questions. What if you simply wanted a box and did not care (or know) what its initial dimensions were? Or, what if you want to be able to initialize a cube by specifying only one value that would be used for all three dimensions? As the `Box` class is currently written, these other options are not available to you.
- ❧ Fortunately, the solution to these problems is quite easy: simply overload the `Box` constructor so that it handles the situations just described

Overloading Constructor: Example

```
class Box {
    double width;
    double height;
    double depth;
    // constructor used when all
    dimensions specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
    // constructor used when no
    dimensions specified
    Box() {
        width = -1; // use -1 to indicate
        height = -1; // an uninitialized
        depth = -1; // box
    }
    // constructor used when cube is
    created
    Box(double len) {
        width = height = depth = len;
    }
    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

```
class OverloadCons {
    public static void main(String args[]) {
        // create boxes using the various
        constructors
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box();
        Box mycube = new Box(7);
        double vol;
        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " +
            vol);
        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " +
            vol);
        // get volume of cube
        vol = mycube.volume();
        System.out.println("Volume of mycube is " +
            vol);
    }
}
```

OUTPUT:

```
Volume of mybox1 is 3000.0
Volume of mybox2 is -1.0
Volume of mycube is 343.0
```

Passing object as a parameter

8

- ❑ So far, we have only been using simple types as parameters to methods.
- ❑ However, it is both correct and common to pass objects to methods.
- ❑ For example, consider the following short program:

Example

1. Define a class '**Test**' with data member **int data**
2. Define a method member
static void changeVal(Test)
which accepts object of class Test as a parameter and changes the value of data for that object
3. In main method create object of class Test, say t1
4. Call the method changeVal() and pass this object as a parameter

NOTE : Object is always “passed by value” in java

Overloading Constructors: Using Objects as Parameters

```
class Test {  
    int a, b;  
    Test(int i, int j) {  
        a = i;  
        b = j;  
    }  
    boolean equalTo(Test o) {  
        if(o.a == a && o.b == b)  
            return true;  
        else return false;  
    }  
}
```

```
class ObjectParamDemo {  
    public static void main(String args[]) {  
        Test ob1 = new Test(100, 22);  
        Test ob2 = new Test(100, 22);  
        Test ob3 = new Test(-1, -1);  
        System.out.println("ob1 == ob2: " +ob1.equalTo(ob2));  
        System.out.println("ob1 == ob3: " +ob1.equalTo(ob3));  
    }  
}
```

Overloading Constructors: Using Objects as Parameters Example

❧ This program generates the following output:

ob1 == ob2: true

ob1 == ob3: false

❧ As you can see, the `equalTo( )` method inside `Test` compares two objects for equality and returns the result. That is, it compares the invoking object with the one that it is passed.

❧ If they contain the same values, then the method returns `true`. Otherwise, it returns `false`.

❧ Notice that the parameter `o` in `equalTo( )` specifies `Test` as its type. Although `Test` is a class type created by the program, it is used in just the same way as Java's built-in types.

❧ One of the most common uses of object parameters involves constructors.

❧ Frequently, you will want to construct a new object so that it is initially the same as some existing object.

Overloading Constructors: Using Objects as Parameters

🔗 Example

```
class Test{  
    int data;  
    Test(){  
    }  
    Test(int d){  
        data=d;  
    }  
    Test(Test obj){  
        data=obj.data;  
    }  
}
```

```
class Main{  
    public static void main(String a[]){  
        Test t1= new Test(5);  
        Test t2=new Test(t1);  
        S.O.P.(t1.data);  
        S.O.P.(t2.data);  
    }  
}
```

Method Overloading

14

- ❑ Multiple methods with same name but different arguments (type and/or number) defined in the same class
- ❑ Used to define methods which perform similar type of operations
- ❑ Lets us reuse same method in a class but with different arguments

Overloading Methods

- ❧ In Java, it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. When this is the case, the methods are said to be overloaded, and the process is referred to as method overloading.
- ❧ Method overloading is one of Java's most exciting and useful features. ***When an overloaded method is invoked, Java uses the type and/or number of arguments as its guide to determine which version of the overloaded method to actually call.*** Thus, overloaded methods must differ in the type and/or number of their parameters.
- ❧ While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method. When Java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.

Rules of Method Overloading

Rules of method overloading ✓

- .Overloaded methods MUST change argument list ✓
- .Overloaded methods ~~can~~ change the ~~return type~~
- .Overloaded methods ~~can~~ change the access modifier
- .Overloaded ~~methods~~ can declare new or broader checked exceptions
- .Overloaded methods have nothing to do with polymorphism
- .Overloading can be done in superclass and subclass methods

Does return type matter in method overloading???

.void sum (int a, int b)

.int sum (int a, int b)

.Defined in same class

.Allowed????...

Does access modifier matter in method overloading???

.private void sum (int a, int b)

.public void sum (int a, int b)

.Defined in same class

.Allowed????...

Example 1

```
public void changeSize(int size, String name, float pattern) { }
```

The following methods are legal overloads of the `changeSize()` method:

```
public void changeSize(int size, String name) { }  
private int changeSize(int size, float pattern) { }  
public void changeSize(float pattern, String name)  
                throws IOException { }
```

Overloading Methods: Example 2

```
class OverloadDemo {  
    void test() {  
        System.out.println("No parameters");  
    }  
    // Overload test for one integer  
parameter.  
    void test(int a) {  
        System.out.println("a: " + a);  
    }  
    // Overload test for two integer  
parameters.  
    void test(int a, int b) {  
        System.out.println("a and b: " + a + " "  
+ b);  
    }  
    // Overload test for a double parameter  
    double test(double a) {  
        System.out.println("double a: " + a);  
        return a*a;  
    }  
}
```

```
class Overload {  
    public static void main(String args[]) {  
        OverloadDemo ob = new OverloadDemo();  
        double result;  
        // call all versions of test()  
        ob.test();  
        ob.test(10);  
        ob.test(10, 20);  
        result = ob.test(123.25);  
        System.out.println("Result of ob.test(123.25): "  
+ result);  
    }  
}
```

OUTPUT:

```
No parameters  
a: 10  
a and b: 10 20  
double a: 123.25  
Result of ob.test(123.25): 15190.5625
```

Overloading Methods: Example

Discussion

- ❧ As you can see, `test( )` is overloaded four times.
- ❧ The first version takes no parameters, the second takes one integer parameter, the third takes two integer parameters, and the fourth takes one double parameter. The fact that the fourth version of `test( )` also returns a value is of no consequence relative to overloading, since return types do not play a role in overload resolution.
- ❧ When an overloaded method is called, Java looks for a match between the arguments used to call the method and the method's parameters. However, this match need not always be exact.

TRY YORUSELF

Overloading main method

```
class MainOverload{  
    public static void main(String ar[]){  
        SOP("Inside main 1");  
        main(10);  
    }  
    static void main (int a){  
        SOP("a is =" +a);  
    }  
}
```

Method overloading with type promotion

```
class Demo{
    void sum(int a, long b){ }
    void sum(int a, int b){ }
    void sum(int a, int b, int c){ }
    PSVM(String ar[]){
        Demo d1=new Demo();
        d1.sum(1,2);
    }
}
```

Explanation

- ❧ If more than one method was identified, pick the most specific one.
- ❧ One method is more specific than another if any invocation handled by the first method could be passed on to the other one without a compile-time error

Array

25

- ❑ Array is collection of variables of same data types.
- ❑ Array in java is an object.
- ❑ An array variable references a group of data.
- ❑ The size of an array is fixed.



Creating an Array

26

```
datatype [ ] arrayName ;    // null
```

```
arrayName = new datatype [10] ; //address of 10 elements  
of the datatype
```

```
datatype [ ] arrayName = new datatype [10]
```

```
int [ ] numbers = new int[20];
```

Default Values

27

- When an array is created its elements are assigned the following default values.
 - ▣ 0 for the numeric primitive datatype.
 - ▣ `\u0000` for char datatype
 - ▣ False for Boolean datatype
 - ▣ Null for reference types.

Accessing Array Elements

28

□ USE BRACKET AND INDICES

```
int [ ]numbers = new int[5]; // { 0, 0, 0, 0, 0}
```

```
numbers[0] = 5 ; // { 5, 0, 0, 0, 0}
```

```
numbers[2] = 7 ; // { 5, 0, 7, 0, 0}
```

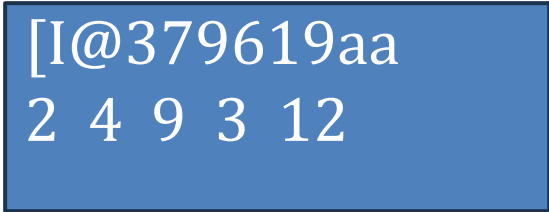
```
numbers[4] = 9 ; // { 5, 0, 7, 0, 9}
```

Printing an Array

29

```
int [ ]numbers = {2, 4, 9, 3,12};  
System.out.println(numbers); // address
```

```
for(int i = 0; i<numbers.length; i++){  
    System.out.println(numbers[i]);  
  
}
```



```
[I@379619aa  
2 4 9 3 12
```

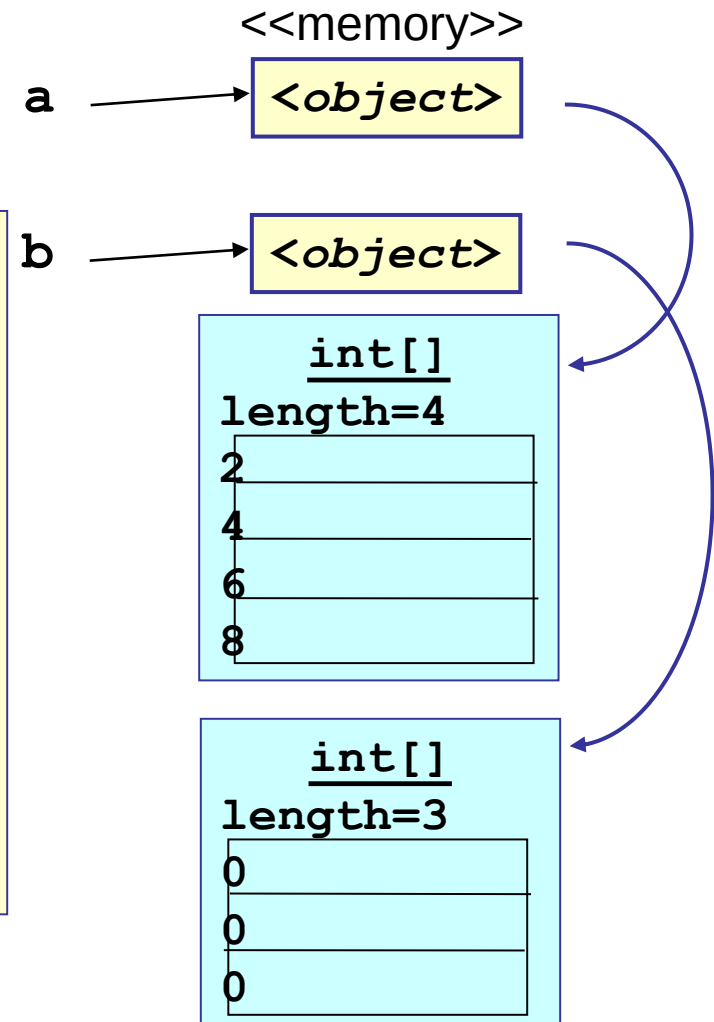
An Array Variable is a *Reference*

30

What does "b = a" do?

What will be printed?

```
int [] a = { 2, 4, 6, 8 };  
int [] b = { 0, 0, 0 };  
b = a;      // Does what?  
b[2] = 999;  
System.out.println(a[2]);  
System.out.println(  
    "b.length=" + b.length );
```

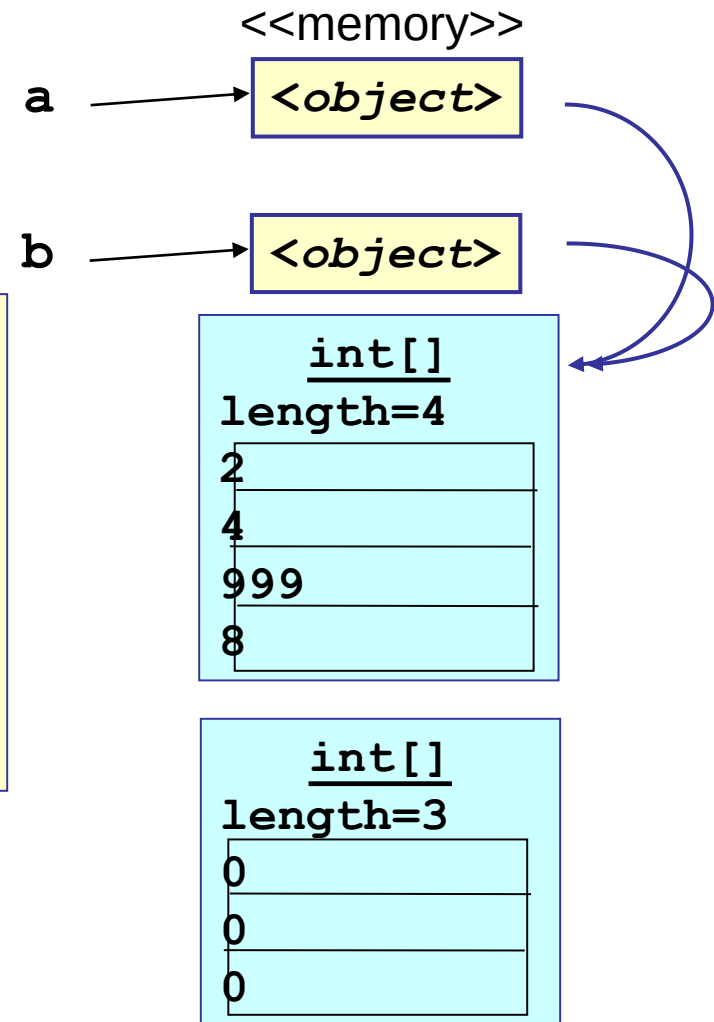


"b = a" copies the *reference*, not the array

31

b = a;
makes **b** refer to same array as **a**.

```
b = a;  
b[2] = 999;  
System.out.println(a[2]);  
System.out.println(  
    "b.length=" + b.length );
```



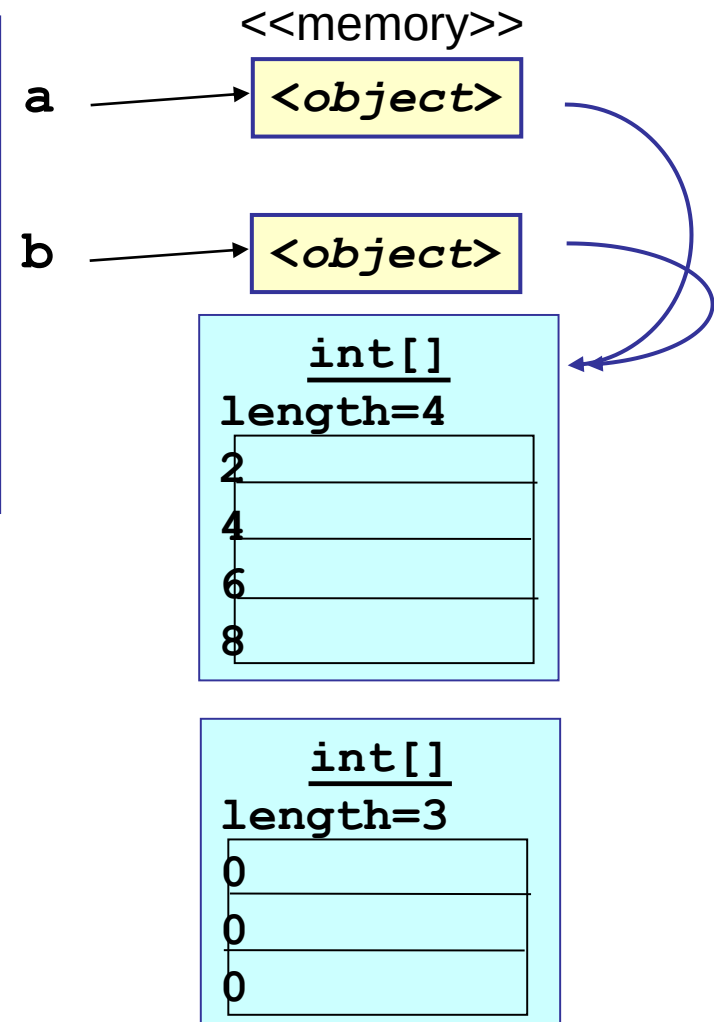
The result:

32

```
b = a;  
b[2] = 999;  
System.out.println(a[2]);  
System.out.println(  
    "b.length=" + b.length );
```

999

b.length = 4



How do you *really* copy an array?

33

Here is one solution:

```
int[] a = { 2, 4, 6, 8 };  
  
// java.util.Arrays.copyOf( ... )  
// creates a new array for copy.  
int[] b = Arrays.copyOf( a, a.length );
```

See also: `System.arraycopy( ... )`

Really Copying An Array

34

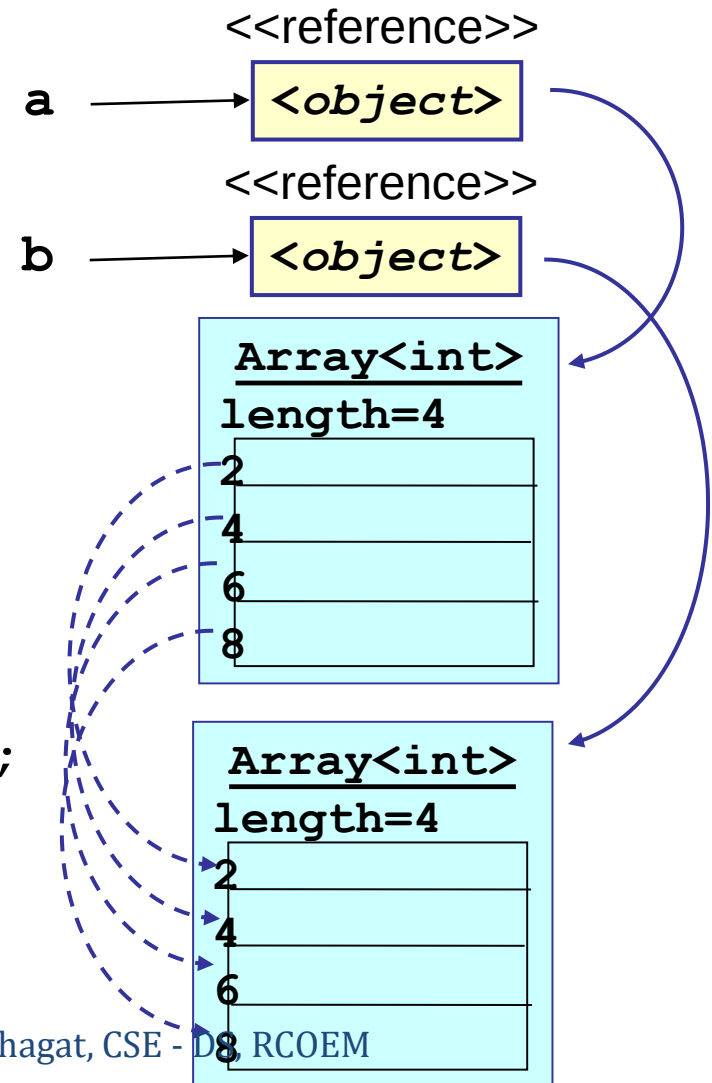
To copy the **contents** of an array, you must copy each element.

```
// copy each element of array
for(int k=0; k<a.length; k++)
b[k] = a[k];
```

This copies all elements of a.

There is an easier way:

```
System.arraycopy(a, 0, b, 0, 4);
```



System.arraycopy

35

System.arraycopy(**src**, src_start, **dest**, dest_start, length)

copies elements from **src** array to **dest** array.
It copies **length** elements only.

```
// copy each element of array  
System.arraycopy(a, 0, b, 0, a.length );
```

"foreach" version of "for" loop

36

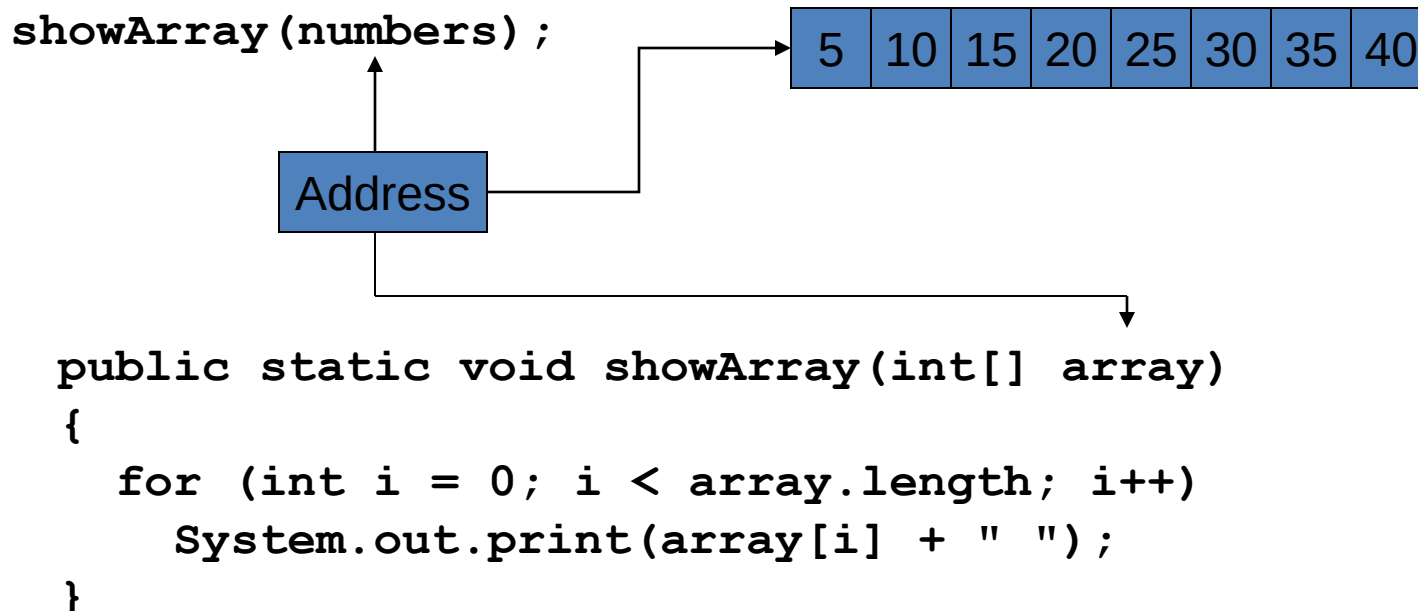
To iterate over every element of an array you can use a special "for" syntax.

```
int [] array = { 2, 4, 6, 8, 10 };  
  
// Iterate over all elements in array.  
// "for each x in array do ..."  
  
for(int x: array) System.out.println( x );
```

Passing Arrays as Arguments

37

- Arrays are objects.
- Their references can be passed to methods like any other object reference variable.



Comparing Arrays

38

- ❑ The == operator determines only whether array references point to the same array object.
- ❑ To compare the contents of an array:

```
int[] firstArray = { 2, 4, 6, 8, 10 };
int[] secondArray = { 2, 4, 6, 8, 10 };
boolean arraysEqual = true;
int i = 0;
if (firstArray.length != secondArray.length)
    arraysEqual = false;
while (arraysEqual && i < firstArray.length)
{
    if (firstArray[i] != secondArray[i])
        arraysEqual = false;
    i++;
}
if (arraysEqual)
    System.out.println("The arrays are equal.");
else
    System.out.println("The arrays are not equal.");
```

Useful Array Operations

39

□ Finding the Highest Value

```
int [] numbers = new int[50];
int highest = numbers[0];
for (int i = 1; i < numbers.length; i++)
{
    if (numbers[i] > highest)
        highest = numbers[i];
}
```

□ Finding the Lowest Value

```
int lowest = numbers[0];
for (int i = 1; i < numbers.length; i++)
{
    if (numbers[i] < lowest)
        lowest = numbers[i];
}
```

Useful Array Operations

40

□ Summing Array Elements:

```
int total = 0; // Initialize accumulator
for (int i = 0; i < units.length; i++)
    total += units[i];
```

□ Averaging Array Elements:

```
double total = 0; // Initialize accumulator
double average; // Will hold the average
for (int i = 0; i < scores.length; i++)
    total += scores[i];
average = total / scores.length;
```

Returning an Array Reference

41

- ❑ A method can return a reference to an array.
- ❑ The return type of the method must be declared as an array of the right type.

```
public static double[] getArray()  
{  
    double[] array = { 1.2, 2.3, 4.5, 6.7, 8.9 };  
    return array;  
}
```

- ❑ The `getArray` method is a public static method that returns an array of doubles.

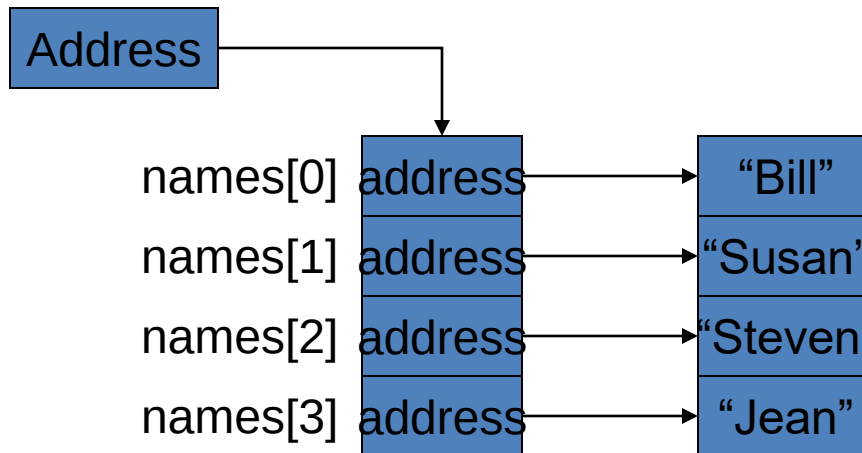
String Arrays

42

- Arrays are not limited to primitive data.
- An array of `String` objects can be created:

```
String[] names = { "Bill", "Susan", "Steven", "Jean" };
```

The `names` variable holds the address to the array. A `String` array is an array of references to `String` objects.

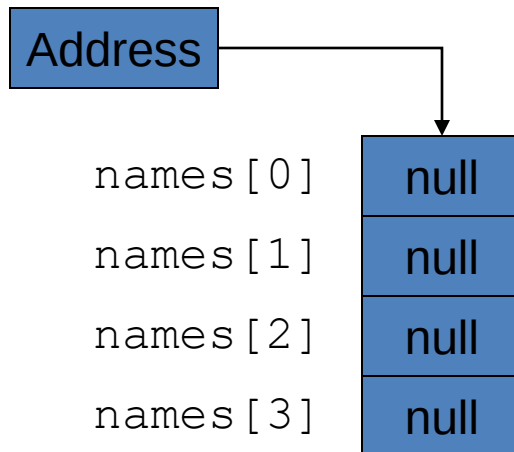


String Arrays

43

- If an initialization list is not provided, the `new` keyword must be used to create the array: **`String[] names = new String[4];`**

The `names` variable holds the address to the array.



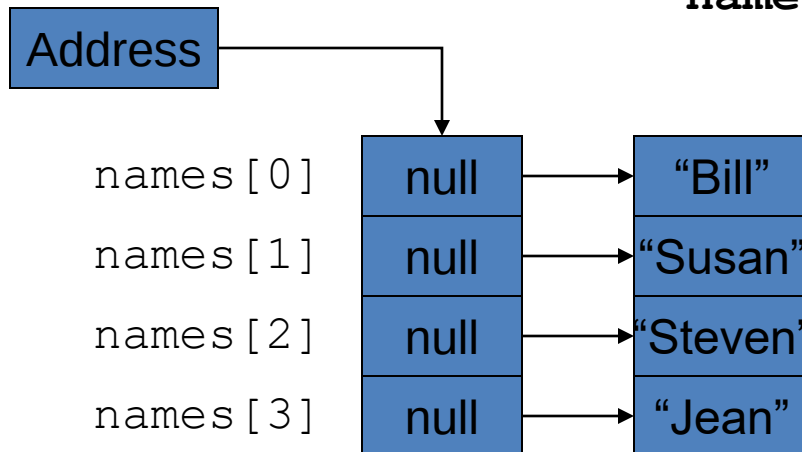
String Arrays

44

- When an array is created in this manner, each element of the array must be initialized.

The `names` variable holds the address to the array.

```
names[0] = "Bill";  
names[1] = "Susan";  
names[2] = "Steven";  
names[3] = "Jean";
```



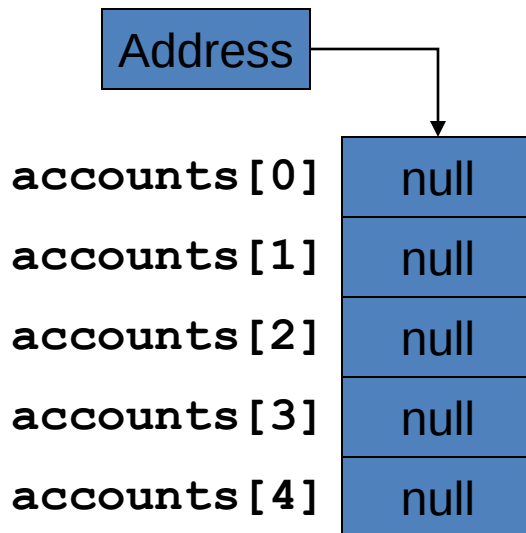
Arrays of Objects

45

- Because `Strings` are objects, we know that arrays can contain objects.

```
BankAccount[] accounts = new BankAccount[5];
```

The `accounts` variable holds the address of an `BankAccount` array.



The array is an array of references to `BankAccount` objects.

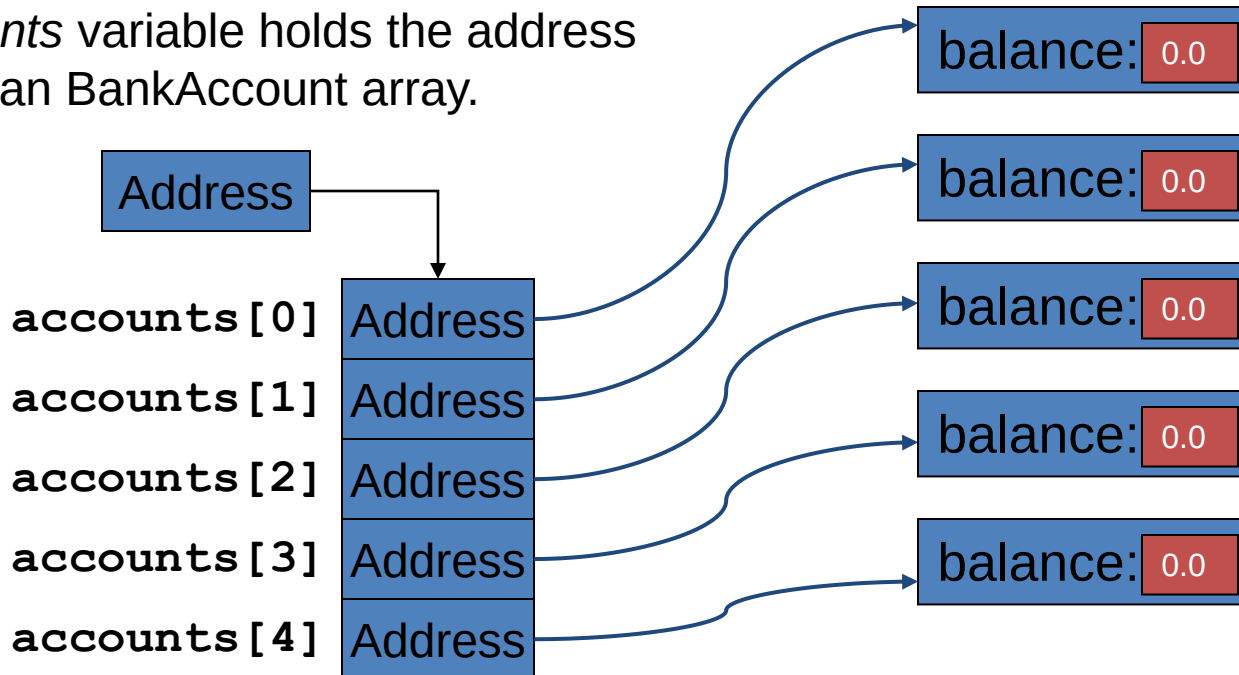
Arrays of Objects

46

- Each element needs to be initialized.

```
for (int i = 0; i < accounts.length; i++)  
    accounts[i] = new BankAccount();
```

The *accounts* variable holds the address of an BankAccount array.



Two-Dimensional Arrays

47

- A two-dimensional array is an array of arrays.
- It can be thought of as having rows and columns.

	column 0	column 1	column 2	column 3
row 0				
row 1				
row 2				
row 3				

Two-Dimensional Arrays

48

- Declaring a two-dimensional array requires two sets of brackets and two size declarators
 - ▣ The first one is for the number of rows
 - ▣ The second one is for the number of columns.

```
double[][] scores = new double[3][4];
```

two dimensional array

rows

columns

- The two sets of brackets in the data type indicate that the scores variable will reference a two-dimensional array.
- Notice that each size declarator is enclosed in its own set of brackets.

Accessing Two-Dimensional Array Elements

49

- When processing the data in a two-dimensional array, each element has two subscripts:
 - ▣ one for its row and
 - ▣ another for its column.

Accessing Two-Dimensional Array Elements

50

The `scores` variable holds the address of a 2D array of `doubles`.

Address		column 0	column 1	column 2	column 3
row 0		scores[0][0]	scores[0][1]	scores[0][2]	scores[0][3]
row 1		scores[1][0]	scores[1][1]	scores[1][2]	scores[1][3]
row 2		scores[2][0]	scores[2][1]	scores[2][2]	scores[2][3]

Accessing Two-Dimensional Array Elements

51

Accessing one of the elements in a two-dimensional array requires the use of both subscripts.

The `scores` variable holds the address of a 2D array of `doubles`.

`scores[2][1] = 95;`

Address		column 0	column 1	column 2	column 3
row 0		0	0	0	0
row 1		0	0	0	0
row 2		0	95	0	0

Accessing Two-Dimensional Array Elements

52

- Programs that process two-dimensional arrays can do so with nested loops.

- To fill the scores array:

```
for (int row = 0; row < 3; row++)  
{  
    for (int col = 0; col < 4; col++)  
    {  
        System.out.print("Enter a score: ");  
        scores[row][col] = keyboard.nextDouble();  
    }  
}
```

Number of rows, not
the largest subscript

Number of
columns, not the
largest subscript

keyboard references
a Scanner object

Accessing Two-Dimensional Array Elements

53

- To print out the `scores` array:

```
for (int row = 0; row < 3; row++)  
{  
    for (int col = 0; col < 4; col++)  
    {  
        System.out.println(scores[row][col]);  
    }  
}
```

Initializing a Two-Dimensional Array

54

- Initializing a two-dimensional array requires enclosing each row's initialization list in its own set of braces.

```
int[][] numbers = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };
```

- Java automatically creates the array and fills its elements with the initialization values.
 - ▣ row 0 {1, 2, 3}
 - ▣ row 1 {4, 5, 6}
 - ▣ row 2 {7, 8, 9}
- Declares an array with three rows and three columns.

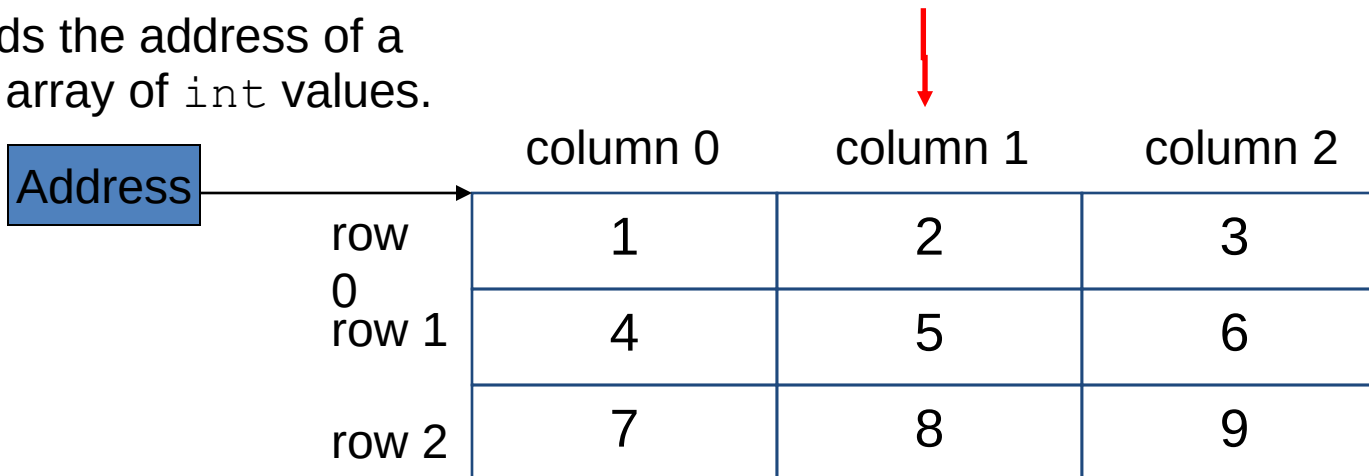
Initializing a Two-Dimensional Array

55

```
int[][] numbers = {{1, 2, 3},  
                  {4, 5, 6},  
                  {7, 8, 9}};
```

produces:

The `numbers` variable holds the address of a 2D array of `int` values.



	column 0	column 1	column 2
row 0	1	2	3
row 1	4	5	6
row 2	7	8	9

The length Field

56

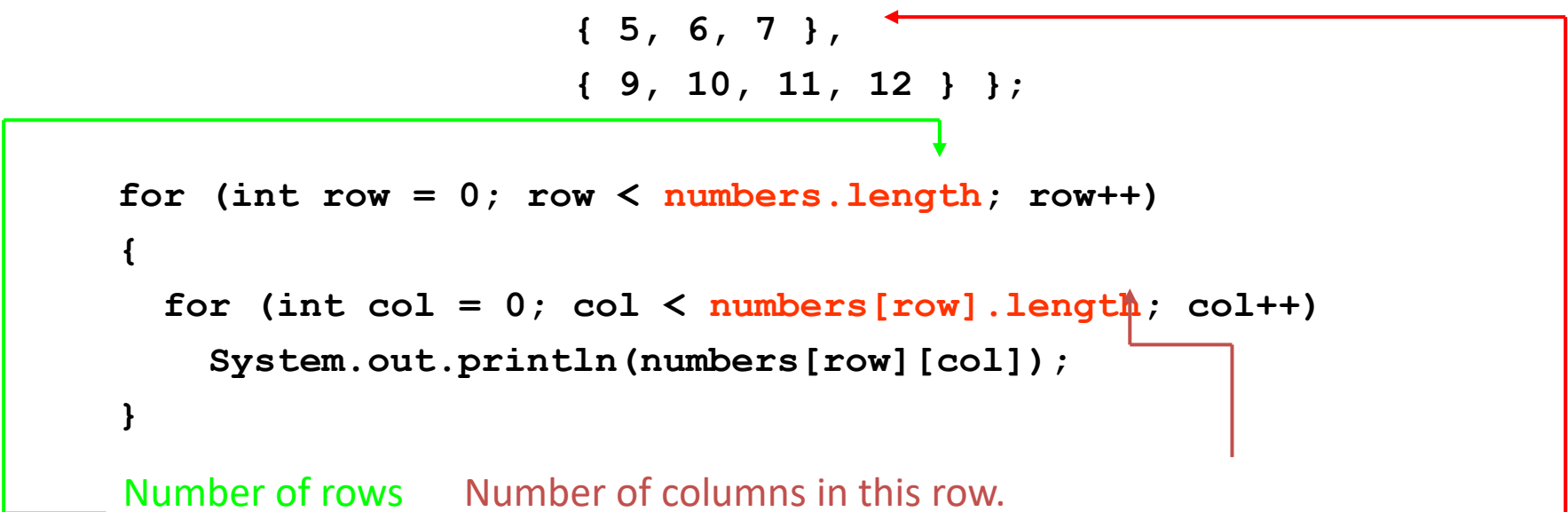
- ❑ Two-dimensional arrays are arrays of one-dimensional arrays.
- ❑ The length field of the array gives the number of rows in the array.
- ❑ Each row has a length constant tells how many columns is in that row.
- ❑ Each row can have a different number of columns.

The length Field

57

- To access the length fields of the array:

```
int[][] numbers = { { 1, 2, 3, 4 },  
                    { 5, 6, 7 },  
                    { 9, 10, 11, 12 } };
```



```
for (int row = 0; row < numbers.length; row++)  
{  
    for (int col = 0; col < numbers[row].length; col++)  
        System.out.println(numbers[row][col]);  
}
```

Number of rows

Number of columns in this row.

The array can have variable length rows.

Summing The Elements of a Two-Dimensional Array

58

```
int[][] numbers = { { 1, 2, 3, 4 },
                    {5, 6, 7, 8},
                    {9, 10, 11, 12} };

int total;
total = 0;
for (int row = 0; row < numbers.length; row++)
{
    for (int col = 0; col < numbers[row].length; col++)
        total += numbers[row][col];
}

System.out.println("The total is " + total);
```

Summing The Rows of a Two-Dimensional Array

59

```
int[][] numbers = {{ 1, 2, 3, 4},
                   {5, 6, 7, 8},
                   {9, 10, 11, 12}};

int total;

for (int row = 0; row < numbers.length; row++)
{
    total = 0;
    for (int col = 0; col < numbers[row].length; col++)
        total += numbers[row][col];
    System.out.println("Total of row "
                       + row + " is " + total);
}
```

Summing The Columns of a Two-Dimensional Array

60

```
int[][] numbers = {{1, 2, 3, 4},
                   {5, 6, 7, 8},
                   {9, 10, 11, 12}};

int total;

for (int col = 0; col < numbers[0].length; col++)
{
    total = 0;
    for (int row = 0; row < numbers.length; row++)
        total += numbers[row][col];
    System.out.println("Total of column "
                       + col + " is " + total);
}
```

Passing and Returning Two-Dimensional Array References

61

- There is no difference between passing a single or two-dimensional array as an argument to a method.
- The method must accept a two-dimensional array as a parameter.

Ragged Arrays

62

- When the rows of a two-dimensional array are of different lengths, the array is known as a *ragged array*.
- You can create a ragged array by creating a two-dimensional array with a specific number of rows, but no columns.

```
int [][] ragged = new int [4][];
```

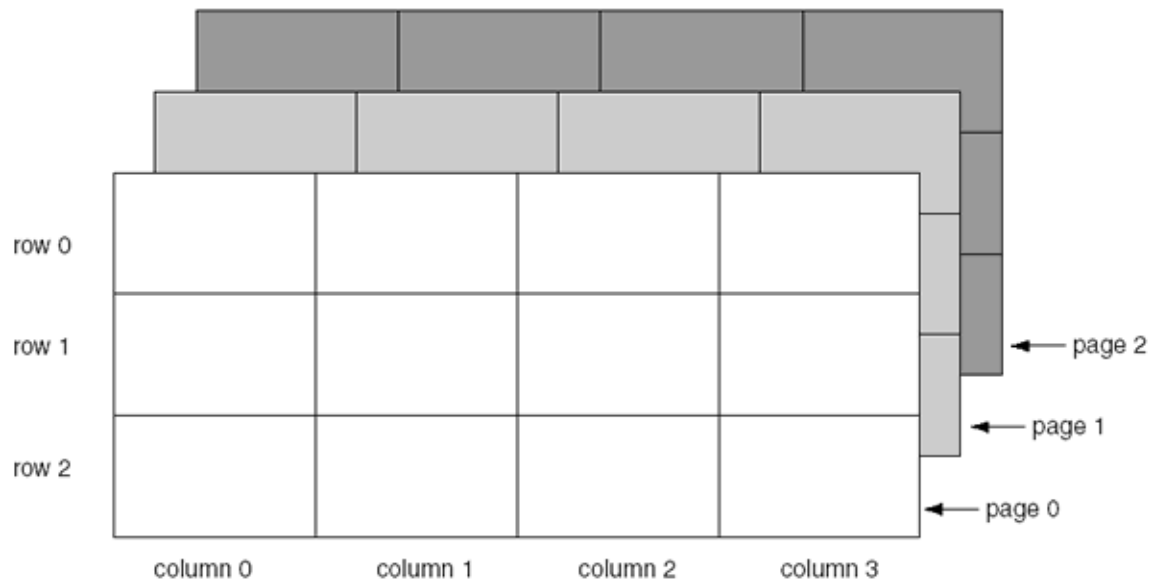
- Then create the individual rows.

```
ragged[0] = new int [3];  
ragged[1] = new int [4];  
ragged[2] = new int [5];  
ragged[3] = new int [6];
```

More Than Two Dimensions

63

- ❑ Java does not limit the number of dimensions that an array may be.
- ❑ More than three dimensions is hard to visualize, but can be useful in some programming problems.



Structure of memory

64

- The fundamental unit of memory is called a **bit**, either 0 or 1.
- In most modern architectures, the smallest unit on which the hardware operates is a sequence of eight consecutive bits called **byte**.

a binary (executable) file

0 1 2 3

010110011000010010011110110000011...

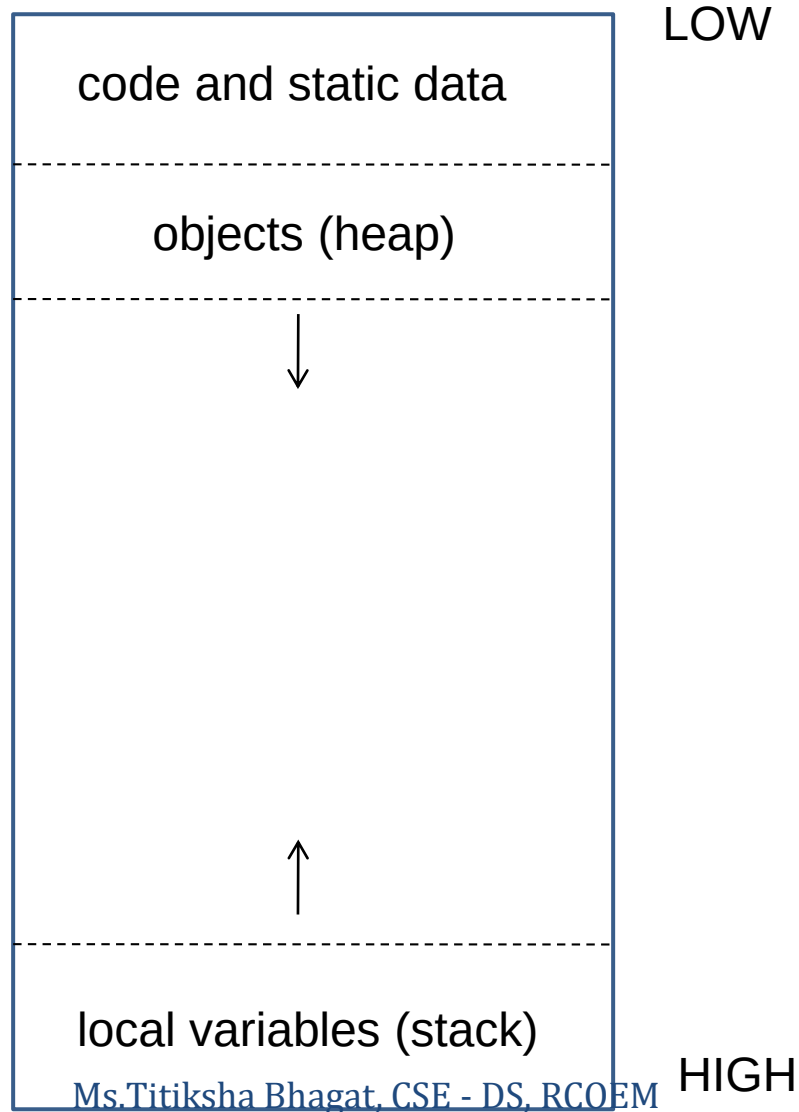
- Numbers and instructions are stored in still larger units. The most common integer size on a particular hardware is called a **word**. Because machines have different architectures, the number of bytes and the order of bytes in a word may vary from machine to machine.

Numbers, bases, and conversion

65

- $21_{10} = 10101_2$
- $0.65625_{10} = 0.10101_2$
- Octal (0,1,2,3,4,5,6,7)
 $10101_2 = 010101_2 = 25_8$
 $0.10101_2 = 0.101010_2 = 0.52_8$
- Hexadecimal (0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F)
 $10101_2 = 00010101_2 = 15_{16}$
 $0.10101_2 = 0.10101000_2 = 0.A8_{16}$
- Useful numbers
 $10000000000_2 = 1024_{10}$ (about 1K)

Memory allocation of variables



Memory allocation to variables

67

- ❑ One region of memory is reserved for **static data**, variables that are never created or destroyed as the program runs, such as named constants and other class variables.
- ❑ When a new object is created, Java allocates space from a pool of memory called the **heap**.
- ❑ Each time a method is called, Java allocates a new block of memory called a stack frame to hold its local variables. When the method returns, its stack frame is erased. Stack frames come from a region of memory called the **stack**.
- ❑ In classical architectures, the stack and heap grow toward each other for flexibility.

Heap-stack diagrams

68

- ❑ It is easier to understand how Java works if you have a good mental model of its use of memory.
- ❑ Whenever your program creates a new object, you need to add a block of memory to the heap. That block must be large enough to store the instance variables for the object, along with some extra space called overhead, that is required for any object.
- ❑ Whenever your program calls a method, you need to create a new stack frame by adding a block of memory to the stack region. The stack frame must be large enough to store local variables for the method, along with some overhead. When a method returns, Java reclaims the memory in its frame.

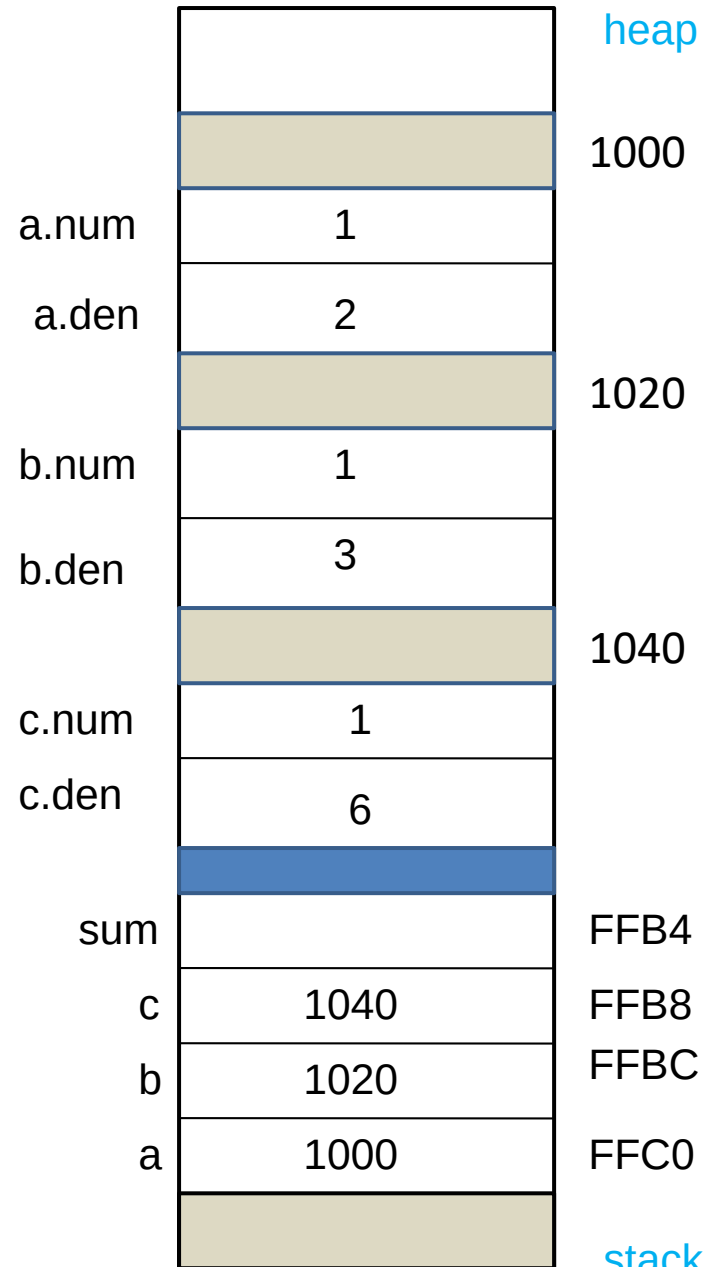
Object references

69

- ❑ Internally, Java identifies an object by its address in memory. That address is called a reference.
- ❑ As an example, when Java evaluates the declaration
 Rational a = new Rational(1, 2)
it allocates heap space for the new Rational object. For this example, imagine that the object is allocated at address 1000.
- ❑ The local variable a is allocated in the current stack frame and is assigned the value (address), which identifies the object.

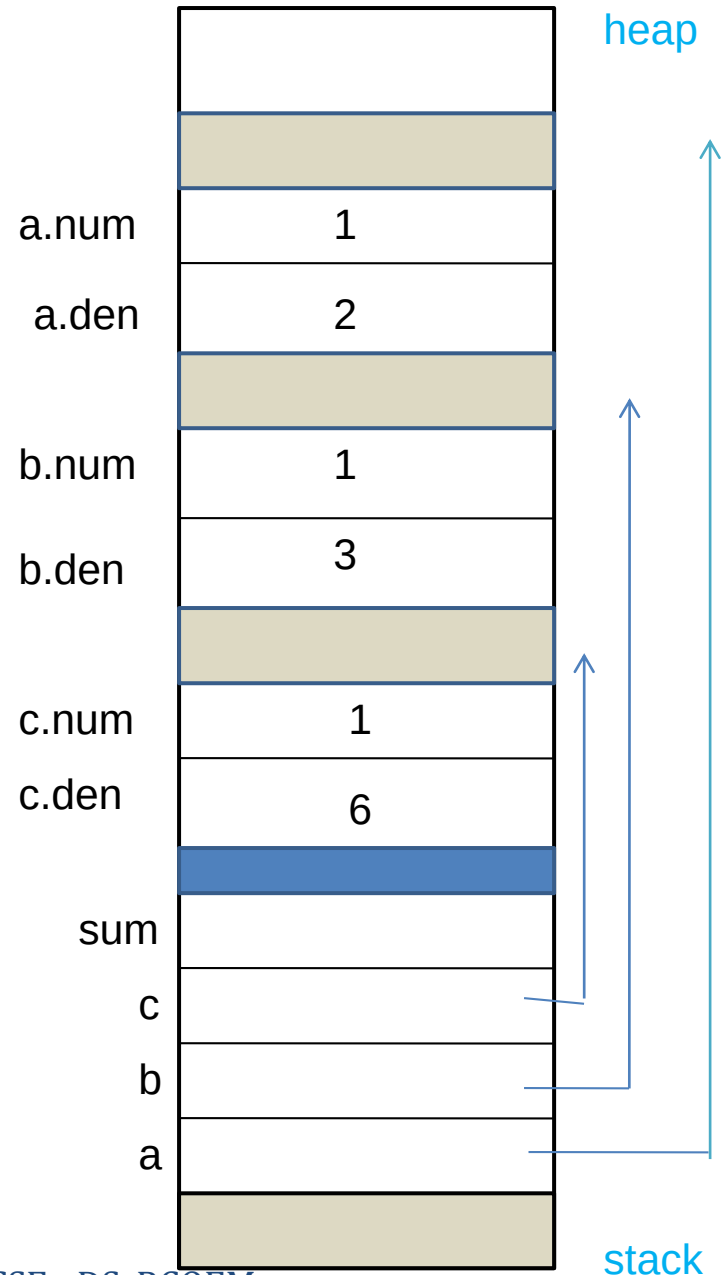
Address model

```
public void run() {
    Rational a = new Rational(1, 2);
    Rational b = new Rational(1, 3);
    Rational c = new Rational(1, 6);
    Rational sum = (a.add(b)).add(c);
}
```



Pointer model

```
public void run() {  
    Rational a = new Rational(1, 2);  
    Rational b = new Rational(1, 3);  
    Rational c = new Rational(1, 6);  
    Rational sum = (a.add(b)).add(c);  
}
```



```

public void run() {
    Rational a = new Rational(1, 2);
    Rational b = new Rational(1, 3);
    Rational c = new Rational(1, 6);
    Rational sum = (a.add(b)).add(c);
}

```

```

public Rational add(Rational r) {
    return new Rational(this.num*r.den
                        + r.num*this.den,
                        this.den*r.den);
}

```

r	1020	FFA8
this	1000	FFAC
sum		FFB4
c	1040	FFB8
b	1020	FFBC
a	1000	FFC0

Stack

		heap
		1000
a.num	1	
a.den	2	
		1020
b.num	1	
b.den	3	
		1040
c.num	1	
c.den	6	

```

public void run() {
    Rational a = new Rational(1, 2);
    Rational b = new Rational(1, 3);
    Rational c = new Rational(1, 6);
    Rational sum = (a.add(b)).add(c);
}

```

```

public Rational add(Rational r) {
    return new Rational(this.num*r.den
        + r.num*this.den,
        this.den*r.den);
}

```

sum		FFB4
c	1040	FFB8
b	1020	FFBC
a	1000	FFC0

			heap
			1000
a.num	1		
a.den	2		
			1020
b.num	1		
b.den	3		
			1040
c.num	1		
c.den	6		
			1060
	5		
	6		


```

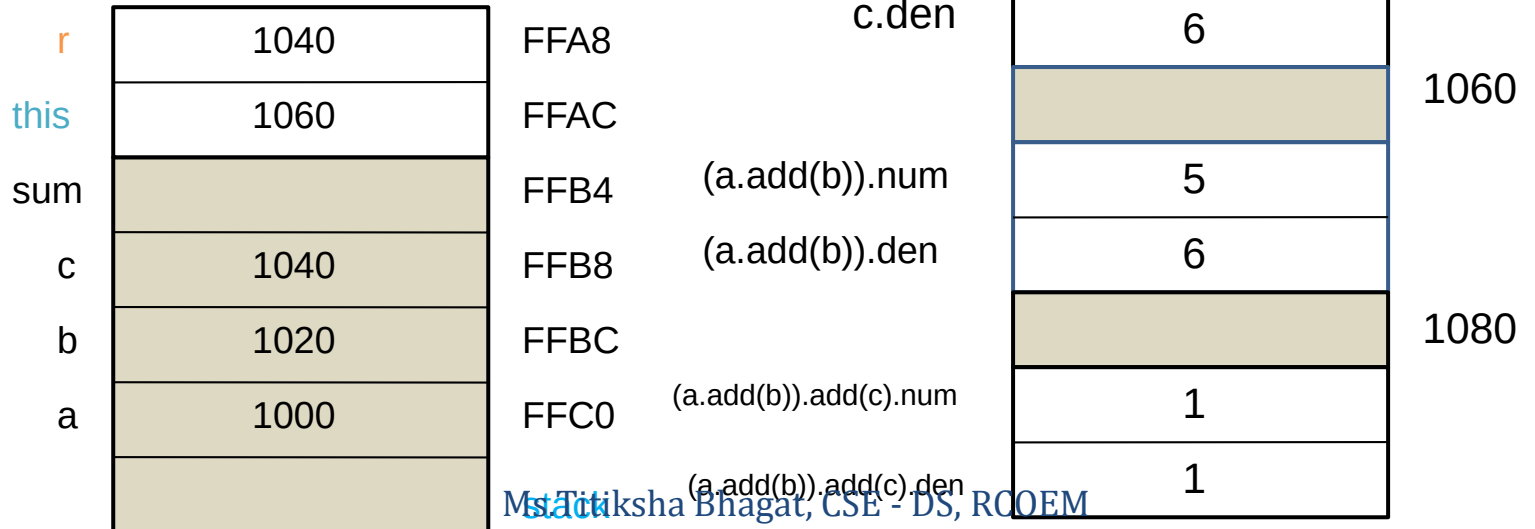
public void run() {
    Rational a = new Rational(1, 2);
    Rational b = new Rational(1, 3);
    Rational c = new Rational(1, 6);
    Rational sum = (a.add(b)).add(c);
}

```

```

public Rational add(Rational r) {
    return new Rational(this.num*r.den
        + r.num*this.den,
        this.den*r.den);
}

```



```

public void run() {
    Rational a = new Rational(1, 2);
    Rational b = new Rational(1, 3);
    Rational c = new Rational(1, 6);
    Rational sum = (a.add(b)).add(c);
}

```

```

public Rational add(Rational r) {
    return new Rational(this.num*r.den
        + r.num*this.den,
        this.den*r.den);
}

```

sum	1080
c	1040
b	1020
a	1000

FFB4	(a.add(b)).num
FFB8	(a.add(b)).den
FFBC	
FFC0	(a.add(b)).add(c).num
	(a.add(b)).add(c).den

			heap
			1000
a.num	1		
a.den	2		
			1020
b.num	1		
b.den	3		
			1040
c.num	1		
c.den	6		
			1060
	5		
	6		
			1080
	1		
	1		

```

public void run() {
    Rational a = new Rational(1, 2);
    Rational b = new Rational(1, 3);
    Rational c = new Rational(1, 6);
    Rational sum = (a.add(b)).add(c);
}

```

```

public Rational add(Rational r) {
    return new Rational(this.num*r.den
        + r.num*this.den,
        this.den*r.den);
}

```

sum	1080
c	1040
b	1020
a	1000

FFB4 (a.add(b)).num

FFB8 (a.add(b)).den

FFBC

FFC0 (a.add(b)).add(c).num

(a.add(b)).add(c).den

		heap
		1000
a.num	1	
a.den	2	
		1020
b.num	1	
b.den	3	
		1040
c.num	1	
c.den	6	
		1060
	5	
	6	
		1080
	1	
	1	

Garbage collection

77

- ❑ In the previous example, the object `a.add(b)` was created in the intermediate step but not referenced by the final stack. It is now **garbage**.
- ❑ When memory is running short, Java does garbage collection
 - ▣ Mark the objects referenced by variables on stack or in static storage.
 - ▣ Sweep all objects in the heap, reclaim unmarked objects (garbage).
- ❑ This process is called **garbage collection** (mark-and-sweep collector).

Exercise: Stack-heap diagram

78

```
public class Point {  
    public Point(int x, int y) {  
        cx = x;  
        cy = y;  
    }
```

```
    private int cx;  
    private int cy;  
}
```

```
public class Line {  
    public Line(Point p1, Point p2) {  
        start = p1;  
        finish = p2;  
    }
```

```
    private Point start;  
    private Point finish;  
}
```

```
    public void run() {  
        Point p1 = new Point(0, 0);  
        Point p2 = new Point(200, 200);  
        Line line = new Line(p1, p2);  
    }
```

Draw a heap-stack diagram (pointer model) showing the state of memory just before the `run()` method returns.

Wrapper Classes

79

- The following declaration creates an Integer object which represents the integer 40 as an object

```
Integer age = new Integer(40);
```

- An object of a wrapper class can be used in any situation where a primitive value will not suffice
- For example, some objects serve as containers of other objects
- Primitive values could not be stored in such containers, but wrapper objects could be

```

class HelloWorld {
    public static void main(String[] args) {
        run();
    }

    public static void run() {
        int x = 17;
        increment(x);
        System.out.println("X=" + x);
    }

    private static void increment(int n) {
        n++;
        System.out.println("N=" + n);
    }
}

```

Primitive type versus objects

81

□ Primitive type

```
public void run() {  
    int x = 17;  
    increment(x);  
    println("x = " + x);  
}
```

```
private void increment(int n) {  
    n++;  
    println("n = " + n);  
}
```

Output

n = 18

n = 17

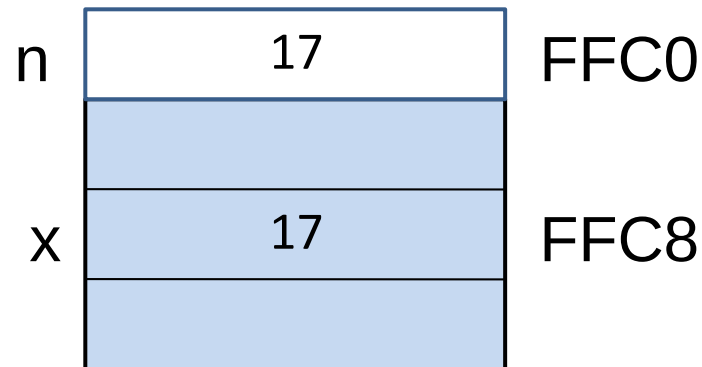
N=18

X=17

When you pass an argument of a primitive type to a method, Java copies the value of the argument into the parameter variable. As a result, changes to the parameter variable have no effect on the argument.

Passing x of primitive type int, a value

increment(x);



x (a value) is copied into n

EmbeddedInteger class

84

```
public class EmbeddedInteger {  
  
    public EmbeddedInteger(int n) {  
        value = n;  
    }  
  
    public void setValue(int n) {  
        value = n;  
    }  
  
    public int getValue() {  
        return value;  
    }  
  
    public String toString() {  
        return "" + value;  
    }  
  
    private int value;  
}
```

Object

```
public void run() {  
    EmbeddedInteger x = new EmbeddedInteger(17);  
    increment(x);  
    println("x = " + x);  
}  
  
private void increment(EmbeddedInteger n) {  
    n.setValue(n.getValue() + 1);  
    println("n = " + n);  
}
```

Output

n = 18

n = 18

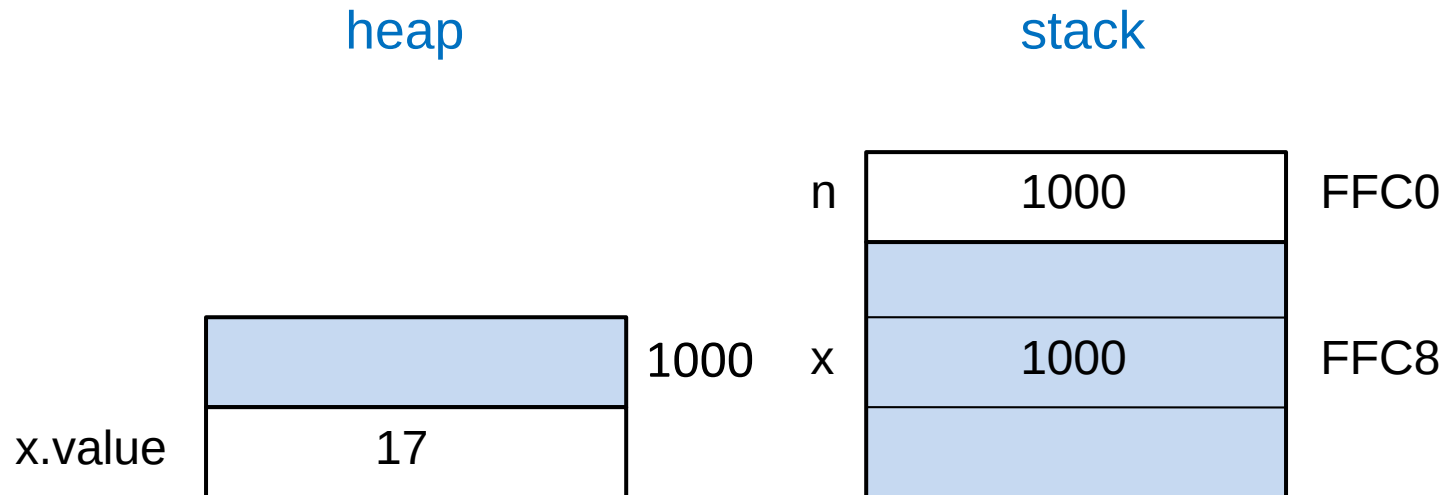
Primitive types vs objects

86

- When you pass an object as an argument, there seems to be some form of sharing going on. However, any changes that you make to the instance variables *inside* an object have a permanent effect on the object.
- Stack-heap diagrams make the reason for this seeming asymmetry clear. When you pass an object to a method, Java copies the *reference*, not the object itself.

Passing object x, a reference (address)

increment(x)



x (a reference to an object) is copied into n
x and n share the same object

Wrapper Classes

3-88

- The java.lang package contains wrapper classes that correspond to each primitive type:

<u>Primitive Type</u>	<u>Wrapper Class</u>
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean
void	Void

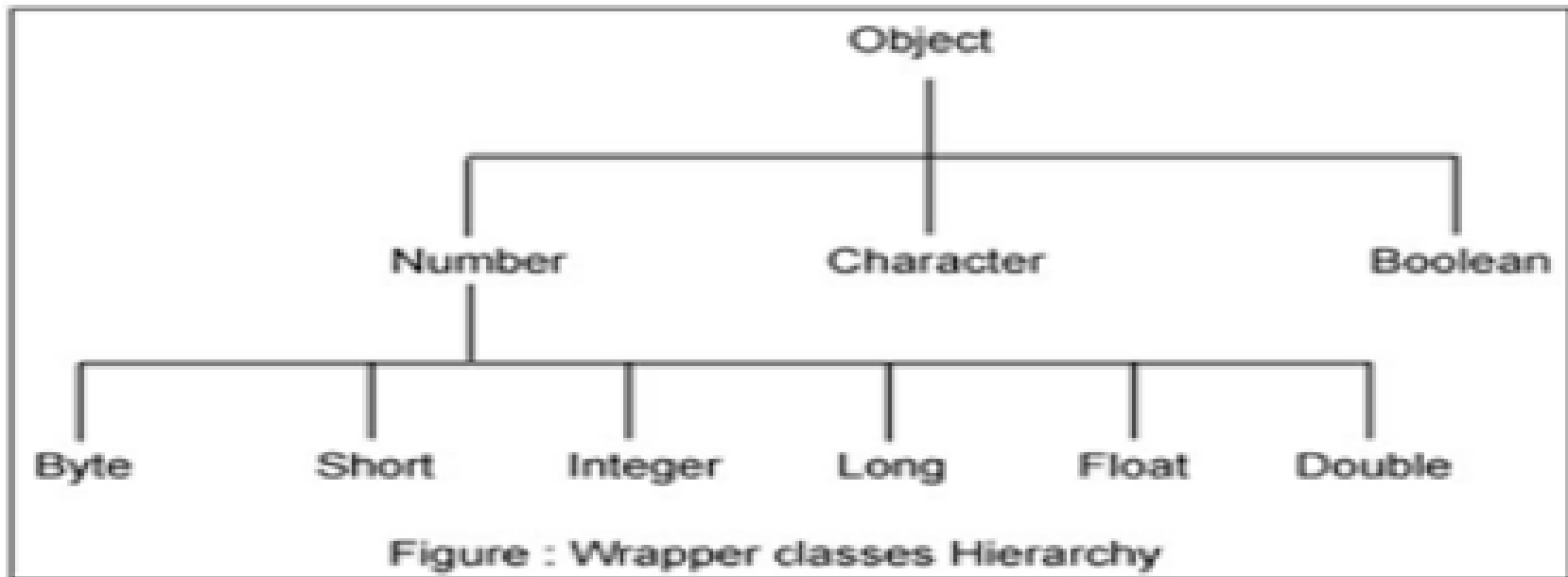
Wrapper Classes

89

- ❑ Each of Java's eight primitive data types has a class dedicated to it.
- ❑ These are known as wrapper classes, because they "wrap" the primitive data type into an object of that class.
- ❑ The wrapper classes are part of the `java.lang` package, which is imported by default into all Java programs.

Wrapper class hierarchy

90



Purpose of Wrapper Classes

91

- The wrapper classes in java serves two primary purposes.
 - ▣ To provide mechanism to 'wrap' primitive values in an object so that primitives can do activities reserved for the objects like being added to ArrayList, HashSet, HashMap etc. collection.
 - ▣ To provide an assortment of utility functions for primitives like converting primitive types to and from string objects, converting to various bases like binary, octal or hexadecimal, or comparing various objects

Object of a wrapper class

92

- ❑ `int x = 25;`
- ❑ declares an int variable named x and initializes it with the value 25.
- ❑ `Integer y = new Integer(33);`
- ❑ Instantiates an Integer object. The object is initialized with the value 33 and a reference to the object is assigned to the object variable y.

Wrapper classes constructor

93

Primitive	Wrapper Class	Constructor Argument
boolean	Boolean	boolean or String
byte	Byte	byte or String
char	Character	char
int	Integer	int or String
float	Float	float, double or String
double	Double	double or String
long	Long	long or String
short	Short	short or String

NumberFormatException

94

- ❑ All wrapper classes (except Character) take String as argument constructor.
- ❑ We might get NumberFormatException if we try to assign invalid argument in constructor.
- ❑ For example to create Integer object we can have following syntax.
- ❑ `Integer intObj = new Integer (25);`
- ❑ `Integer intObj2 = new Integer ("25");`
- ❑ Here in we can provide any number as string argument but not the words etc.

NumberFormatException

95

- ❑ Below statement will throw run time exception (NumberFormatException)
- ❑ `Integer intObj3 = new Integer ("Two");`
- ❑ The EXAMPLE discussion focuses on the Integer wrapperclass, but applies in a general sense to all eight wrapper classes

Using wrapper classes

96

- You can create an instance of a wrapper class by calling its constructor with the primitive value. For example,

```
Integer five = new Integer(5);
```

creates a new Integer object containing the value 5.

- For each of the wrapper classes, Java defines a method to retrieve the primitive value, as illustrated below:

```
int underlyingValue = five.intValue();
```

Autoboxing

97

- Autoboxing is the automatic conversion of a primitive value to a corresponding wrapper object:

```
Integer obj;
```

```
int num = 42;
```

```
obj = num;
```

- The assignment creates the appropriate Integer object
- The reverse conversion (called unboxing) also occurs automatically as needed

Wrapper Classes

98

- ❑ Wrapper classes also contain static methods that help manage the associated type
- ❑ For example, the Integer class contains a method to convert an integer stored in a String to an int value:

```
num = Integer.parseInt(str);
```
- ❑ The wrapper classes often contain useful constants as well
- ❑ The Integer class contains MIN_VALUE and MAX_VALUE which hold the smallest and largest int values

Method	Purpose
<code>parseInt(s)</code>	returns a signed decimal integer value equivalent to string s
<code>toString(i)</code>	returns a new String object representing the integer i
<code>byteValue()</code>	returns the value of this Integer as a byte
<code>doubleValue()</code>	returns the value of this Integer as an double
<code>floatValue()</code>	returns the value of this Integer as a float
<code>intValue()</code>	returns the value of this Integer as an int
<code>shortValue()</code>	returns the value of this Integer as a short
<code>longValue()</code>	returns the value of this Integer as a long
<code>int compareTo(int i)</code>	Compares the numerical value of the invoking object with that of i. Returns 0 if the values are equal. Returns a negative value if the invoking object has a lower value. Returns a positive value if the invoking object has a greater value.

Boxing and unboxing

100

- As of Java Standard Edition 5.0, Java automatically converts values back and forth between a primitive type and the corresponding wrapper class. For example, if you write

`Integer five = 5;`

Java will automatically call the Integer constructor.

- Similarly, if you then write

`int six = five + 1;`

Java will automatically call `intValue` before the addition.

- These operations are called boxing and unboxing.
- Although boxing and unboxing can be quite convenient, this feature can generate confusion and should be used with care.

String class facts

101

- ❑ An object of the String class represents a string of characters.
- ❑ The String class belongs to the java.lang package, which does not require an import statement.
- ❑ Like other classes, String has constructors and methods.
- ❑ Unlike other classes, String has two operators, + and += (used for concatenation).

Literal Strings

102

- ❑ are anonymous objects of the String class
- ❑ are defined by enclosing text in double quotes.
“This is a literal String”
- ❑ don’t have to be constructed.
- ❑ can be assigned to String variables.
- ❑ can be passed to methods and constructors as parameters.
- ❑ have methods you can call.

Literal String examples

103

```
//assign a literal to a String variable  
String name = "Robert";
```

```
//calling a method on a literal String  
char firstInitial = "Robert".charAt(0);
```

```
//calling a method on a String variable  
char firstInitial = name.charAt(0);
```

Immutability

104

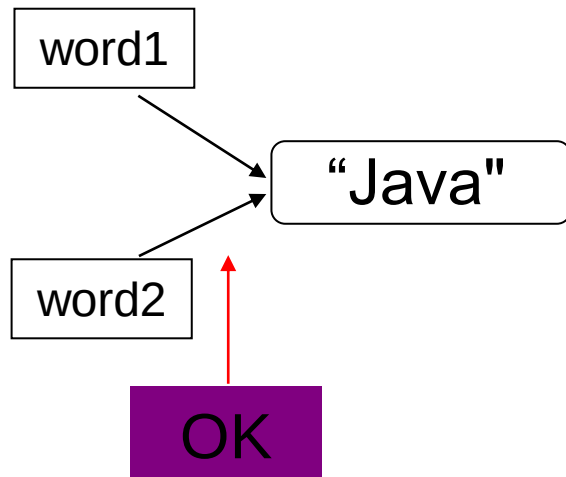
- ❑ Once created, a string cannot be changed: none of its methods changes the string.
- ❑ Such objects are called *immutable*.
- ❑ Immutable objects are convenient because several references can point to the same object safely: there is no danger of changing an object through one reference without the others being aware of the change.

Advantages Of Immutability

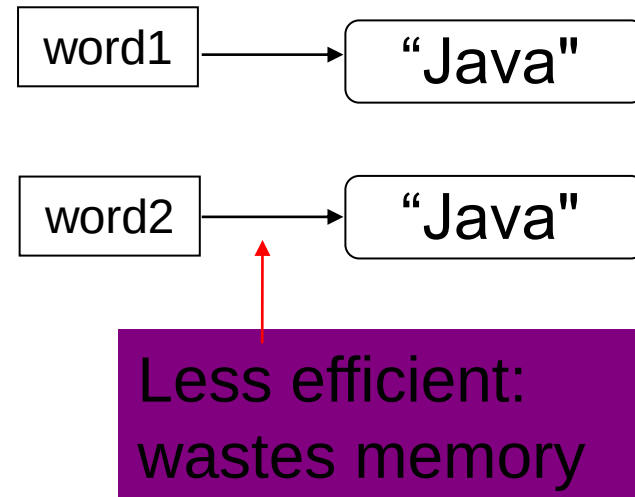
105

Uses less memory.

```
String word1 = "Java";  
String word2 = word1;
```



```
String word1 = "Java";  
String word2 = new String(word1);
```

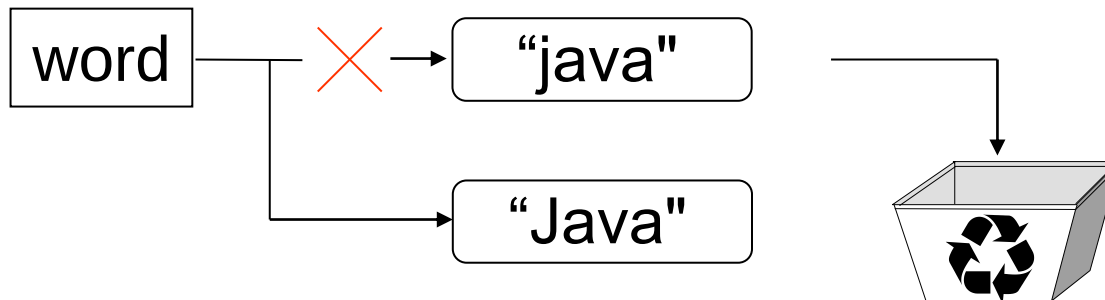


Disadvantages of Immutability

106

Less efficient — you need to create a new string and throw away the old one even for small changes.

```
String word = "Java";  
char ch = Character.toUpperCase(word.charAt (0));  
word = ch + word.substring (1);
```



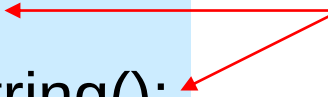
Empty Strings

107

- An empty String has no characters. It's length is 0.

```
String word1 = "";  
String word2 = new String();
```

Empty strings



- Not the same as an uninitialized String.

```
private String errorMsg;
```

errorMsg
is null



No Argument Constructors

108

- No-argument constructor creates an empty String. Rarely used.

```
String empty = new String();
```

- A more common approach is to reassign the variable to an empty literal String. (Often done to reinitialize a variable used to store input.)

```
String empty = ""; //nothing between quotes
```

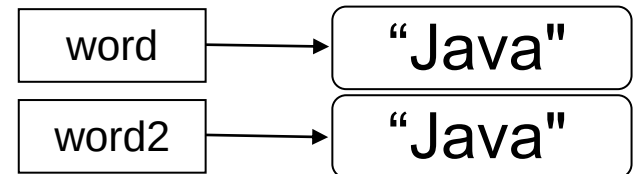
Copy Constructors

109

- ❑ Copy constructor creates a copy of an existing String. Also rarely used.
- ❑ Not the same as an assignment.

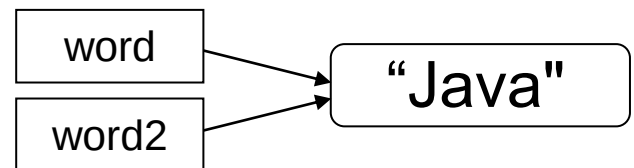
Copy Constructor: Each variable points to a different copy of the String.

```
String word = new String("Java");  
String word2 = new String(word);
```



Assignment: Both variables point to the same String.

```
String word = "Java";  
String word2 = word;
```



Other Constructors

110

Most other constructors take an array as a parameter to create a String.

```
char[] letters = {'J', 'a', 'v', 'a'};  
String word = new String(letters); //"Java"
```

Methods — length, charAt

111

`int length();`

- Returns the number of characters in the string

`char charAt(i);`

- Returns the char at position i.

Character positions in strings are numbered starting from 0 – just like arrays.

Returns:

`"Problem".length();`

`"Window".charAt (2);`

7

'n'

Methods — substring

112

Returns a new String by copying characters from an existing String.

□ `String subs = word.substring(i, k);`

▣ returns the substring of chars in positions from **i** to **k-1**

television
↑ ↑
i *k*

□ `String subs = word.substring(i);`

▣ returns the substring from the **i**-th char to the end

television
↑
i

`"television".substring(2,5);`
`"immutable".substring(2);`
`"bob".substring(9);`

Returns:

→ "lev"
→ "mutable"
→ "" (empty string)

Methods — Concatenation

113

```
String word1 = "re", word2 = "think"; word3 = "ing";  
int num = 2;
```

- ❑ `String result = word1 + word2;`
 `//concatenates word1 and word2 "rethink"`
- ❑ `String result = word1.concat(word2);`
 `//the same as word1 + word2 "rethink"`
- ❑ `result += word3;`
 `//concatenates word3 to result "rethinking"`
- ❑ `result += num; //converts num to String`
 `//and concatenates it to result "rethinking2"`

Methods — Find (indexOf)

114

0 2 6 10 15

String name = "President George Washington";

Returns:

name.indexOf ('P'); 0

name.indexOf ('e'); 2

name.indexOf ("George"); 10

name.indexOf ('e', 3);

(starts searching
at position 3)

name.indexOf ("Bob"); -1

(not found)

name.lastIndexOf ('e'); 15

Methods — Equality

115

`boolean b = word1.equals(word2);`
returns **true** if the string **word1** is equal to **word2**

`boolean b = word1.equalsIgnoreCase(word2);`
returns **true** if the string **word1** matches **word2**,
case-blind

```
b = "Raiders".equals("Raiders");//true
b = "Raiders".equals("raiders");//false
b = "Raiders".equalsIgnoreCase("raiders");//true
```

```
if(team.equalsIgnoreCase("raiders"))
    System.out.println("Go You " + team);
```

Methods — Comparisons

116

`int diff = word1.compareTo(word2);`
returns the “difference” **word1 - word2**

`int diff = word1.compareToIgnoreCase(word2);`
returns the “difference” **word1 - word2**,
case-blind

Usually programmers don't care what the numerical “difference” of **word1 - word2** is, just whether the difference is negative (word1 comes before word2), zero (word1 and word2 are equal) or positive (word1 comes after word2). Often used in conditional statements.

```
if(word1.compareTo(word2) > 0){  
    //word1 comes after word2...  
}
```

Comparison Examples

117

//negative differences

diff = "apple".compareTo("berry");//a before b

diff = "zebra".compareTo("apple");//z before a

diff = "dig".compareTo("dug");//i before u

diff = "dig".compareTo("digs");//dig is shorter

//zero differences

diff = "apple".compareTo("apple");//equal

diff = "dig".compareToIgnoreCase("DIG");//equal

//positive differences

diff = "berry".compareTo("apple");//b after a

diff = "apple".compareTo("Apple");//a after A

diff = "BIT".compareTo("BIG");//T after G

diff = "huge".compareTo("hug");//huge is longer

Methods — trim

118

`String word2 = word1.trim ();`
returns a new string formed from **word1** by
removing white space at both ends
does not affect whites space in the middle

```
String word1 = " Hi Bob ";  
String word2 = word1.trim();  
//word2 is "Hi Bob" – no spaces on either end  
//word1 is still " Hi Bob " – with spaces
```

Methods — replace

119

String word2 = word1.**replace**(oldCh, newCh);
returns a new string formed from **word1** by
replacing all occurrences of **oldCh** with **newCh**

```
String word1 = "rare";  
String word2 = "rare".replace('r', 'd');  
//word2 is "dade", but word1 is still "rare"
```

Methods — Changing Case

120

```
String word2 = word1.toUpperCase();
```

```
String word3 = word1.toLowerCase();
```

returns a new string formed from **word1** by
converting its characters to upper (lower) case

```
String word1 = "HeLLo";
```

```
String word2 = word1.toUpperCase();//"HELLO"
```

```
String word3 = word1.toLowerCase();//"hello"
```

```
//word1 is still "HeLLo"
```

Replacements

121

- Example: to “convert” word1 to upper case, replace the reference with a new reference.

```
word1 = word1.toUpperCase();
```

- A common bug:

```
word1.toUpperCase();
```

word1
remains
unchanged

Packages

122

- ❑ Packages are Java's way of grouping a number of related classes and/or interfaces together into a single unit. That means, packages act as “containers” for classes.
- ❑ The benefits of organising classes into packages are:
 - The classes contained in the packages of other programs/applications can be reused.
 - In packages classes can be unique compared with classes in other packages. That two classes in two different packages can have the same name. If there is a naming clash, then classes can be accessed with their fully qualified name.
 - Classes in packages can be hidden if we don't want other packages to access them.
 - Packages also provide a way for separating “design” from coding.

Java Foundation Packages

123

Java provides a large number of classes grouped into different packages based on their functionality

□ The six foundation Java packages are:

- java.lang :

Contains classes for primitive types, strings, math functions, threads, and exception.

- java.util :

Contains classes such as vectors, hash tables, date etc.

- java.io :

Stream classes for I/O .

- java.awt :

Classes for implementing GUI – windows, buttons, menus etc.

- java.net :

Classes for networking .

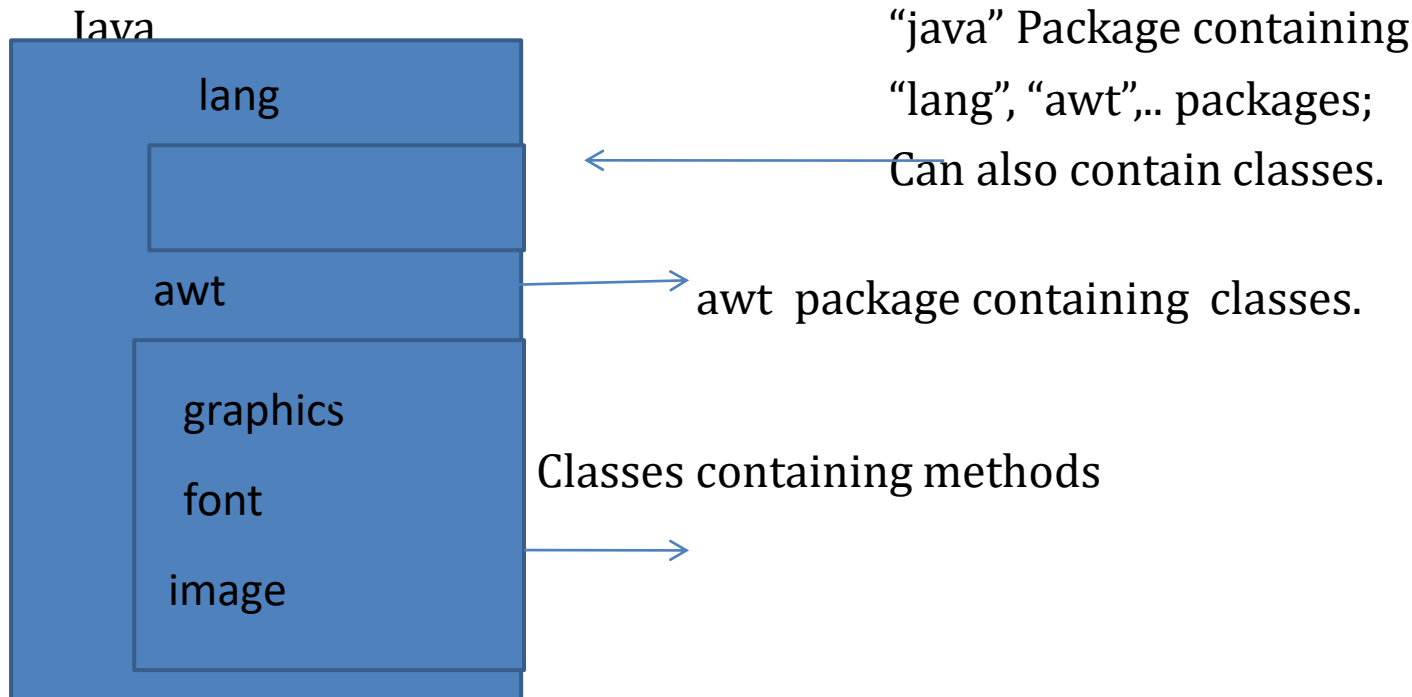
- java.applet :

Classes for creating and implementing app.

Using System Packages

124

- The packages are organised in a hierarchical structure .For example, a package named “java” contains the package “awt”, which in turn contains various classes required for implementing GUI (graphical user interface).



Accessing Classes from Packages

125

There are two ways of accessing the classes stored in packages:

- Using fully qualified class name

```
java.lang.Math.sqrt(x);
```

- Import package and use class name directly

```
import java.lang.Math
```

```
Math.sqrt(x);
```

- Selected or all classes in packages can be imported:

```
import package.class;  
import package.*;
```

- Implicit in all programs: `import java.lang.*;`
- package statement(s) must appear first

Creating Packages

126

Java supports a keyword called “package” for creating user-defined packages. The package statement must be the first statement in a Java source file (except comments and white spaces) followed by one or more classes.

```
package myPackage;  
public class ClassA {  
    // class body  
}  
class ClassB {  
}
```

Package name is “myPackage” and classes are considered as part of this package; The code is saved in a file called “ClassA.java” and located in a directory called “myPackage”.

Accessing a Package

127

- As indicated earlier, classes in packages can be accessed using a fully qualified name or using a short-cut as long as we import a corresponding package.
- The general form of importing package is:
`import package1[.package2][...].classname`
- Example:
`import myPackage.ClassA;`
`import myPackage.secondPackage`
- All classes/packages from higher-level package can be imported as follows:
`import myPackage.*;`

Using a Package

128

- Let us store the code listing below in a file named “ClassA.java” within subdirectory named “myPackage” within the current directory (say “abc”).

- package myPackage;
- public class ClassA {
- // class body
- public void display()
- {
- System.out.println("Hello, I am ClassA");
- }
- }
- class ClassB {
- // class body
- }

Using a Package

129

Within the current directory (“abc”) store the following code in a file named “ClassX.java”

```
import myPackage.ClassA;
public class ClassX
{
    public static void main(String args[])
    {
        ClassA objA = new ClassA();
        objA.display();
    }
}
```


Compiling and Running

130

- When ClassX.java is compiled, the compiler compiles it and places .class file in current directory. If .class of ClassA in subdirectory “myPackage” is not found, it compiles ClassA also.
- Note: It does not include code of ClassA into ClassX †
- When the program ClassX is run, java loader looks for ClassA.class file in a package called “myPackage” and loads it.

Using a Package

131

Let us store the code listing below in a file named “ClassA.java” within subdirectory named “secondPackage” within the current directory (say “abc”).

```
public class ClassC {  
    // class body  
    public void display()  
    {  
        System.out.println("Hello, I am ClassC");  
    }  
}
```

Using a Package

132

- ❑ Within the current directory (“abc”) store the following code in a file named “ClassX.java”
- ❑ `import myPackage.ClassA;`
- ❑ `import secondPackage.ClassC;`
- ❑ `public class ClassY`
- ❑ `{`
- ❑ `public static void main(String args[])`
- ❑ `{`
- ❑ `ClassA objA = new ClassA();`
- ❑ `ClassC objC = new ClassC();`
- ❑ `objA.display();`
- ❑ `objC.display();`
- ❑ `}`
- ❑ `}`

Output

133

- Hello, I am ClassA
- Hello, I am ClassC

Package

134

- ⑩ There are folders or directories in our computers for the classification and accessibility of various files, and in Java, we have packages for the same
- ⑩ In Java, Packages are similar to folders, which are mainly used to organize classes and interfaces
- ⑩ Java provides some built-in packages which we can use but we can also create our own (user-defined) packages
- ⑩ Packages are used in Java in order to prevent naming conflicts, to control access, to make searching/locating and usage of classes, interfaces, enumerations and annotations easier, etc.
- ⑩ *Packages* are containers for classes that are used to keep the class name space compartmentalized.
- ⑩ For example, a package allows you to create a class named **List**, which you can store in your own package without concern that it will collide with some other class named **List** stored elsewhere.
- ⑩ Packages are stored in a hierarchical manner and are explicitly imported into new class definitions.

Packages are used for

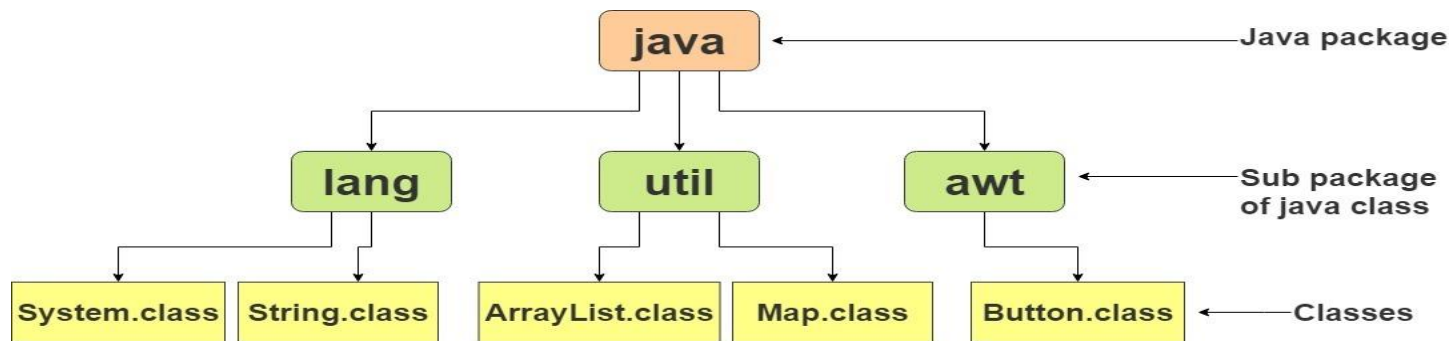
135

- **Packages in Java is a mechanism to encapsulate a group of classes, sub packages and Interfaces.**
- **Preventing naming conflicts.** For example there can be two classes with name Teacher in two packages, college.staff.icse.Teacher and college.staff.cbse.Teacher
- **Making searching/locating** and usage of classes, interfaces, enumerations and annotations easier
- **Providing access control:** protected and default have package level access control. A protected member is accessible by classes in the same package and its subclasses. A default member (without any access specifier) is accessible by classes in the same package only
- Packages can be considered as **data encapsulation** (or data-hiding)
- **Reusability**
- **Better Organization**

Types of Packages

136

- ⑩ **User defined package:** The package we create is called user-defined package.
- ⑩ **Built-in package:** The already defined package like `java.io.*`, `java.lang.*` etc are known as built-in packages.
- ⑩ for example when we need user input, we import a package like this:



→ and Scanner is a class which is present in the sub package util.

1. `java.lang.*` : Defines actual language.Imported automatically. Includes classes for primitive types, strings, math function, threads and exceptions
2. `java.util.*` : Datastructures provided like hash tables, queue,etc.
3. `java.io.*` : Input/Output support classes for data I/O
4. `java.net.*` : Networking-classes for communicating with computers over the internet
5. `java.awt.*` : Windows in java. GUI classes for buttons, list, menus, etc.
6. `javax.swing.*` : GUI java's version of windows
7. `java.sql.*` : Database transactions
8. `java.net.http.*` : web programming
9. `java.applet.*` : Classes for creating and implementing applets

Defining a Package

138

- To create a package, include a package command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package.
- The package statement defines a name space in which classes are stored. If you omit the package statement, the class names are put into the default package, which has no name.
- While the default package is fine for short, sample programs, it is inadequate for real applications. Most of the time, you will define a package for your code.
- This is the general form of the package statement:

□ `package pkg;`

□ Here, *pkg* is the name of the package.

- For example, the following statement creates a package called MyPackage.

□ `package MyPackage;`

Creating user defined packages

139

10 Syntax

```
package<name of package>; //package declaration
class <class name>{
                                //class definition
                                //Body of class
}
```

- .java file must be saved in the directory having the name of specified package
- On compilation .class created in same directory

• Example

```
package mypack;
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to package");
    }
}
```

Package Hierarchy

Package firstpackage.secondpackage

- Allows similar classes to be grouped into same package
- Similar packages to be grouped into a larger packages

Ex. java.io.Writer

Accessing package

```
import package1.package2.....classname
```

Or

```
import packagename.*
```

Package Example

```
package package1;
```

```
public class MyDate{
```

```
    private int dd,mm,yy;
```

```
    public MyDate(){
```

```
        dd=1;mm=2;yy=2010;
```

```
    }
```

```
    public void showDate(){
```

```
        System.out.println(dd+"/"+mm+"/"+yy);
```

```
    }
```

```
}
```

```
import package1.MyDate;
```

```
class PackageDemo2{
```

```
    public static void main(String arg[]){
```

```
        MyDate d1 = new MyDate();
```

```
        MyDate d1 = new MyDate();
```

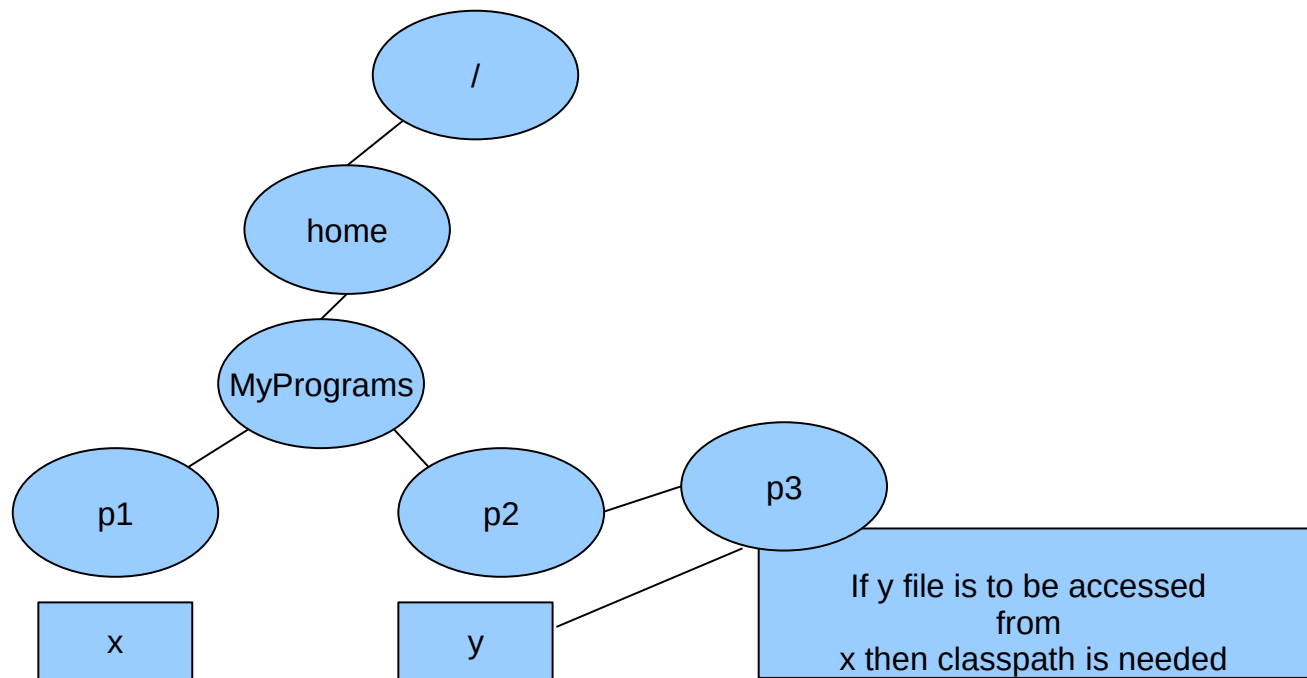
```
        d1.showDate();
```

```
        d2.showDate();
```

```
    }
```

```
}
```

Directory structure & classpath



Creating a class inside package while importing another package

144

□ The order in this case should be:

⑩ → package declaration

⑩ → package import

//Declaring a package

package anotherpackage;

//importing a package

import letmecalculate.Calculator;

public class Example{

 public static void main(String args[]){

 Calculator obj = new Calculator();

 System.out.println(obj.add(100, 200));

 }

Using fully qualified name while importing a class

145

10 Calculator.java

```
package letmecalculate;
public class Calculator {
    public int add(int a, int b){
        return a+b;
    }
    public static void main(String args[]){
        Calculator obj = new Calculator();
        System.out.println(obj.add(10,
20));
    }
}
```

10 Example.java

```
//Declaring a package
package anotherpackage;
public class Example{
    public static void main(String args[]){
        //Using fully qualified name instead of
import
        letmecalculate.Calculator obj = new
        letmecalculate.Calculator();
        System.out.println(obj.add(100, 200));
    }
}
```

Subpackages:

146

- ⑩ Packages that are inside another package are the subpackages
- ⑩ These are not imported by default, they have to be imported explicitly
- ⑩ If you import a package, all the classes and interfaces of that package will be imported excluding the classes and interfaces of the subpackages
- ⑩ Hence, you need to import the subpackage as well
- ⑩ Let's say I have created another package inside calculate and the subpackage name is multiply.
- ⑩ So if I create a class in this subpackage it should have this package declaration in the beginning:
 - package calculate.multiply;

Default Access Modifier

147

- ⑩ The scope of this modifier is limited to the package only
- ⑩ This means that if we have a class with the default access modifier in a package, only those classes that are in this package can access this class
- ⑩ No other class outside this package can access this class
- ⑩ Similarly, if we have a default method or data member in a class, it would not be visible in the class of another package

Default Access Modifier Example

```
package abcpackage;

public class Addition {
    /* Since we didn't mention any access modifier
    here, it would
    * be considered as default.
    */
    int addTwoNumbers(int a, int b){
        return a+b;
    }
}
```

OUTPUT: Exception in thread "main" java.lang.Error:
Unresolved compilation problem:
The method addTwoNumbers(int, int) from the type
Addition is not visible
at xyzpackage.Test.main(Test.java:12)

```
package xyzpackage;

/* We are importing the abcpackage
* but still we will get error because the
* class we are trying to use has default access
* modifier.
*/
import abcpackage.*;

public class Test {
    public static void main(String args[]){
        Addition obj = new Addition();
        /* It will throw error because we are trying to access
        * the default method in another package
        */
        obj.addTwoNumbers(10, 21);
    }
}
```

Protected Access Modifier

149

- ⑩ Protected data member and method are only accessible by the classes of the same package and the subclasses present in any package
- ⑩ The protected access modifier is similar to default access modifier with one exception that it has visibility in sub classes
- ⑩ Classes cannot be declared protected
- ⑩ This access modifier is generally used in a parent child relationship

Protected Access Modifier - Example

150

⑩ In this example the class Test which is present in another package is able to call the addTwoNumbers() method, which is declared protected

```
package abcpackage;
```

⑩ This is because the Test class extends class Addition and the protected modifier allows the access of protected members in subclasses (in any packages)

```
public class Addition {
```

```
    protected int addTwoNumbers(int a, int b)
```

```
{
```

```
        return a+b;
```

```
}
```

```
}
```

```
package xyzpackage;  
import abcpackage.*;
```

```
class Test extends Addition{
```

```
    public static void main(String args[]){
```

```
        Test obj = new Test();
```

```
        System.out.println(obj.addTwoNumbers(11, 22));
```

```
    }
```

```
}
```

Public Access Modifier

151

- ⑩ The members, methods and classes that are declared public can be accessed from anywhere
- ⑩ This modifier doesn't put any restriction on the access

```
package ABCpackage;
```

```
public class Addition {
```

```
    public int addTwoNumbers(int a, int b){  
        return a+b;  
    }  
}
```

```
package XYZpackage;  
import ABCpackage.*;
```

```
public class test {
```

```
    public static void main(String args[]){  
        Addition obj = new Addition();  
        System.out.println(obj.addTwoNumbers(100,  
1));  
    }  
}
```